discrete-math: the Discrete Math program — user guide

discrete-math-ai999x 1.1.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Discrete Math covers number theory, modular arithmetic, combinatorics, graph theory and network algorithms, logic and the mathematics of cryptography. This guide is about `discrete-math`, the model as a self-contained program for Windows and Linux: 99 capabilities in 7 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/discrete-math-ai999x/); the Python package (`pip install discrete-math-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Discrete Math	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-discrete-math-1.4.0-windows-x86_64.zip
Desktop app: Discrete Math	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-discrete-math-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Discrete Math in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-discrete-math-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-discrete-math.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-discrete-math-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-discrete-math-1.4.0-linux-x86_64.tar.gz
./ai999x-discrete-math-1.4.0-linux-x86_64/ai999x-discrete-math
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `discrete-math` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source
~/.bashrc
discrete-math --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
discrete-math --version
```

In your browser

Open https://www.ai999x.com/models/discrete-math-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install discrete-math-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ discrete-math --version
discrete-math-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ discrete-math ask "Find the inverse of 3 modulo 11."
Read as: Modular multiplicative inverse
inverse = 4
  by: Modular multiplicative inverse  (extended Euclidean algorithm)
  assumes: m >= 2; gcd(a, m) = 1 (otherwise no inverse exists)
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ discrete-math search "modular multiplicative inverse"
mod_inverse                         Modular multiplicative inverse  (modular arithmetic /
inverses)
multiplicative_order                Multiplicative order  (modular arithmetic / orders and
primitive roots)
mod_pow                             Modular exponentiation  (modular arithmetic / powers)
linear_congruence                   Linear congruence ax ≡ b (mod m)  (modular arithmetic /
congruences)
sqrt_mod                            Square roots modulo a prime  (number theory / quadratic
residues)
mod_reduce                          Remainder modulo m (Euclidean division)  (modular arithmetic /
residues)
primitive_root                      Smallest primitive root modulo a prime  (modular arithmetic /
orders and primitive roots)
crt                                 Chinese remainder theorem  (modular arithmetic / congruences)
… 12 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ discrete-math list
affine_cipher                          Affine cipher
articulation_points                    Articulation points (cut vertices)
base_convert                           Base (radix) conversion
bell                                   Bell numbers
bernoulli                              Bernoulli numbers
bfs_order                              Breadth-first search order
binomial_term                          Coefficient in a binomial expansion
bipartite                              Bipartiteness test
… 91 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ discrete-math info mod_inverse
Modular multiplicative inverse
  extended Euclidean algorithm
  in   a        the integer to invert (integer) [1]
  in   m        the modulus (at least 2) (integer) [1]
  out  inverse  a^{-1} mod m [1]
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ discrete-math solve mod_inverse a=3 m=11
inverse = 4
  by: Modular multiplicative inverse  (extended Euclidean algorithm)
  assumes: m >= 2; gcd(a, m) = 1 (otherwise no inverse exists)
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ discrete-math solve mod_inverse a=3 m=11 --json
{"outcome":"SOLVED","message":"","value":4.0,"unit":"","solved_for":"inverse","values":
[["inverse",4,""]],"target":"inverse","phrase":"a^{-1} mod m","assumptions":["m >= 2","gcd(a, m) =
1 (otherwise no inverse exists)"],"name":"Modular multiplicative inverse","equation":"extended
Euclidean algorithm"}
```

2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ discrete-math solve mod_inverse a=3
NEEDS_INPUT: mod_inverse needs m (the modulus (at least 2)). Example: solve('mod_inverse', a=3,
m=11).
```

2.9 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ discrete-math figures mod_inverse a=3 m=11 --out .
[
  {
    "slug": "mod-inverse-live-1",
    "title": "A^{-1} mod m as the integer to invert varies",
    "caption": "A^{-1} mod m as a runs from 1 to 7, every other input as you entered it (7
solutions). Marked: your a = 3 gives inverse = 4. Your inputs: a = 3, m = 11.",
    "file": ".\\mod-inverse-live-1.svg"
  },
  {
    "slug": "mod-inverse-live-2",
    "title": "A^{-1} mod m as the modulus (at least 2) varies",
    "caption": "A^{-1} mod m as m runs from 2 to 28, every other input as you entered it (11
solutions). Values in this range where the inputs admit no answer (4 of 15) are left out. Marked:
your m = 11 gives inverse = 4. Your inputs: a = 3, m = 11.",
    "file": ".\\mod-inverse-live-2.svg"
  }
]
```

2.10 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | discrete-math batch

`cases.jsonl`:

```
{"key": "mod_inverse", "inputs": {"a": "3", "m": "11"}}
{"key": "crt", "inputs": {"residues": "[2, 3, 2]", "moduli": "[3, 5, 7]"}}
```

```
$ discrete-math batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":4.0,"unit":"","solved_for":"inverse","values":
[["inverse",4,""]],"target":"inverse","phrase":"a^{-1} mod m","assumptions":["m >= 2","gcd(a, m) =
1 (otherwise no inverse exists)"],"name":"Modular multiplicative inverse","equation":"extended
Euclidean algorithm"}
{"outcome":"SOLVED","message":"","value":23.0,"unit":"","solved_for":"x","values":[["x",23,""],
["modulus",105,""]],"target":"x","phrase":"the smallest non-negative solution","assumptions":["one
residue per modulus","the system is consistent (r_i ≡ r_j modulo gcd(m_i, m_j))"],"name":"Chinese
remainder theorem","equation":"merge the congruences pairwise with the extended Euclidean algorithm
(works for non-coprime moduli)"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `discrete-math info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`discrete-math solve ... --json` prints one answer as one line of JSON. `discrete-math batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | discrete-math batch`. `discrete-math catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]

answer field	meaning
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["discrete-math", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`discrete-math figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

```
$ discrete-math figures mod_inverse a=3 m=11 --out .
[
  {
    "slug": "mod-inverse-live-1",
    "title": "A^{-1} mod m as the integer to invert varies",
    "caption": "A^{-1} mod m as a runs from 1 to 7, every other input as you entered it (7
solutions). Marked: your a = 3 gives inverse = 4. Your inputs: a = 3, m = 11.",
    "file": ".\\mod-inverse-live-1.svg"
  },
  {
    "slug": "mod-inverse-live-2",
    "title": "A^{-1} mod m as the modulus (at least 2) varies",
    "caption": "A^{-1} mod m as m runs from 2 to 28, every other input as you entered it (11
solutions). Values in this range where the inputs admit no answer (4 of 15) are left out. Marked:
your m = 11 gives inverse = 4. Your inputs: a = 3, m = 11.",
    "file": ".\\mod-inverse-live-2.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `discrete_math_ai999x.wasm` (https://www.ai999x.com/compute/rust/discrete_math_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("discrete_math_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `discrete-math info <id>` shows the same inputs and outputs in the terminal.

Combinatorics (25)

Bell numbers — `bell`

B_n: the number of partitions of an n-element set into any number of non-empty blocks.

$$B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k$$

symbol		meaning	SI unit
`n`	in	size of the set (0..200)	—
`bell`	out	the Bell number B_n	—

Applies when: 0 <= n <= 200.

```
$ discrete-math solve bell n=5
bell = 52
  by: Bell numbers  (Bell triangle (Aitken's array))
  assumes: 0 <= n <= 200
```

What the worked example means: This calculation determines the total number of ways to partition a set containing 5 elements into any number of non-empty subsets. The result, denoted as 52, represents the sum of all Stirling numbers of the second kind for that specific set size. It quantifies the comprehensive possibilities for splitting the group without regard to the order of blocks or their internal arrangement.

Bernoulli numbers — `bernoulli`

The Bernoulli number B_n as an exact fraction (convention B_1 = -1/2).

$$\sum_{k=0}^{m} \binom{m+1}{k} B_k = 0, \quad B_0 = 1$$

symbol		meaning	SI unit
`n`	in	the index n (0..200)	—
`value`	out	B_n as a decimal	—
`numerator`	out	numerator of B_n in lowest terms	—
`denominator`	out	denominator of B_n (positive)	—

Applies when: 0 <= n <= 200; B_1 = -1/2 (the 'first' Bernoulli numbers).

```
$ discrete-math solve bernoulli n=10
numerator = 5
denominator = 66
value = 0.07575757576
  by: Bernoulli numbers  (exact recurrence sum_{k=0}^{m} C(m+1, k) B_k = 0 in rational arithmetic)
  assumes: 0 <= n <= 200; B_1 = -1/2 (the 'first' Bernoulli numbers)
```

What the worked example means: This calculation determines the Bernoulli number corresponding to the input index 10. Using a recursive definition with rational arithmetic, the result is expressed as an exact fraction where the numerator is 5 and the denominator is 66. The decimal approximation of this value is 0.0757576. This method ensures precision by avoiding floating-point errors during the computation.

Bracelets (rotations and reflections identified) — `bracelets`

Distinct bracelets of n beads in k colours when rotations and flips count as the same.

$$B(n,k) = \tfrac{1}{2}N(n,k) + \begin{cases} \tfrac{1}{2}k^{(n+1)/2} & n \text{ odd} \\ \tfrac{1}{4}(k+1)k^{n/2} & n \text{ even} \end{cases}$$

symbol		meaning	SI unit
`beads`	in	number of beads n (1..60)	—
`colours`	in	number of colours k (1..60)	—
`bracelets`	out	classes under rotation and reflection	—
`necklaces`	out	classes under rotation only	—

Applies when: 1 <= n, k <= 60.

```
$ discrete-math solve bracelets beads=6 colours=2
bracelets = 13
necklaces = 14
  by: Bracelets (rotations and reflections identified)  (Burnside's lemma over the dihedral group)
  assumes: 1 <= n, k <= 60
```

What the worked example means: This calculation determines the count of unique circular patterns using Burnside's lemma on the dihedral group, where rotations and reflections are considered identical. Given 6 beads and 2 available colours, the model computes that there are 13 distinct bracelet configurations. It also reports 14 as the count when only rotational symmetry is applied.

Catalan numbers — `catalan`

C_n: balanced parenthesizations with n pairs, triangulations of an (n+2)-gon, full binary trees with n+1 leaves.

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

symbol		meaning	SI unit
`n`	in	the index n (0..500)	—
`catalan`	out	the Catalan number C_n	—

Applies when: 0 <= n <= 500; a convex polygon with s sides has C_(s-2) triangulations.

```
$ discrete-math solve catalan n=10
catalan = 16796
  by: Catalan numbers  (closed form binomial(2n, n)/(n + 1))
  assumes: 0 <= n <= 500; a convex polygon with s sides has C_(s-2) triangulations
```

What the worked example means: This calculation determines the nth Catalan number using the closed form formula involving binomial coefficients. Given an input index of 10, the resulting value 16796 represents the count of specific combinatorial structures such as balanced parenthesizations, triangulations of a convex polygon, or full binary trees with a corresponding number of leaves.

Coefficient in a binomial expansion — `binomial_term`

The coefficient of x^k in (a x + b)^n by the binomial theorem (exact integer).

$$(ax+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k} x^k$$

symbol		meaning	SI unit
`a`	in	coefficient of x inside the bracket	—
`b`	in	constant inside the bracket	—
`n`	in	the power n (0..1000)	—
`k`	in	the power of x asked for	—
`coefficient`	out	C(n, k) a^k b^(n-k)	—

Applies when: integer a, b; 0 <= k <= n <= 1000.

```
$ discrete-math solve binomial_term a=2 b=3 n=5 k=3
coefficient = 720
  by: Coefficient in a binomial expansion  (binomial theorem)
  assumes: integer a, b; 0 <= k <= n <= 1000
```

What the worked example means: This calculation determines the exact integer coefficient for the specific power of x requested, using the binomial theorem. It combines the input values for the inner coefficient a, the constant b, and the exponent n to produce the result 720, which represents the multiplier for the term containing x raised to the power 3.

Combinations C(n, k) — `combinations`

The number of ways to choose k of n distinct items when order does not matter (exact integer).

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

symbol		meaning	SI unit
`n`	in	number of items to choose from (0..1000)	—
`k`	in	number chosen (0..n)	—
`combinations`	out	C(n, k) = n! / (k! (n-k)!)	—

Applies when: 0 <= k <= n <= 1000; result at most 10^300.

```
$ discrete-math solve combinations n=10 k=3
combinations = 120
  by: Combinations C(n, k)  (multiplicative formula with exact integer division at every step)
  assumes: 0 <= k <= n <= 1000; result at most 10^300
```

What the worked example means: This calculation determines the count of distinct subsets of size 3 that can be formed from a set containing 10 elements. The result, 120, represents the exact integer value derived using a multiplicative formula with precise division at each step to ensure accuracy and avoid overflow issues common in factorial calculations.

Combinations with repetition (multisets) — `multisets`

The number of ways to choose k items from n types when types may repeat and order does not matter.

$$\left(\binom{n}{k}\right) = \binom{n+k-1}{k}$$

symbol		meaning	SI unit
`n`	in	number of types (at least 1)	—
`k`	in	number of items chosen	—
`multisets`	out	C(n + k - 1, k)	—

Applies when: n >= 1; result at most 10^300.

```
$ discrete-math solve multisets n=5 k=3
multisets = 35
  by: Combinations with repetition (multisets)  (stars and bars)
  assumes: n >= 1; result at most 10^300
```

What the worked example means: This calculation determines the count of ways to choose k items from n types where repetition is allowed and order does not matter. Using the stars and bars method, the result is derived from the binomial coefficient C(n + k - 1, k). The computed value 35 represents the total distinct combinations possible given the input parameters.

Counting multiples by inclusion-exclusion — `inclusion_exclusion`

How many integers in 1..n are divisible by at least one of the given numbers (and how many by none).

$$\left|\bigcup_i A_i\right| = \sum_{\emptyset \neq S} (-1)^{|S|+1} \left\lfloor \frac{n}{\operatorname{lcm}(S)} \right\rfloor$$

symbol		meaning	SI unit
`n`	in	the upper end of the range 1..n	—
`divisors`	in	the divisors (whole numbers >= 1)	—
`divisible_by_any`	out	count divisible by at least one divisor	—
`divisible_by_none`	out	count divisible by none	—

Applies when: n >= 1; 1 to 12 divisors, each >= 1.

```
$ discrete-math solve inclusion_exclusion n=1000 "divisors=[3, 5, 7]"
divisible_by_any = 543
divisible_by_none = 457
  by: Counting multiples by inclusion-exclusion  (inclusion-exclusion over subsets of the divisors,
using floor(n / lcm(subset)))
  assumes: n >= 1; 1 to 12 divisors, each >= 1
```

What the worked example means: This calculation determines how many whole numbers from one up to the upper limit n are divisible by at least one of the specified divisors. It uses the principle of inclusion-exclusion, adding counts for single divisors and subtracting overlaps for pairs and larger groups to avoid double counting. The result shows that 543 numbers share a factor with the given set, leaving 457 numbers in the range that are coprime to all provided divisors.

Derangements — `derangements`

!n: permutations of n items with no item in its original place, and the probability !n/n!.

$$!n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!} = (n-1)\big(!(n-1) + !(n-2)\big)$$

symbol		meaning	SI unit
`n`	in	number of items (0..170)	—
`derangements`	out	!n	—
`probability`	out	!n / n! (tends to 1/e)	—

Applies when: 0 <= n <= 170.

```
$ discrete-math solve derangements n=5
derangements = 44
probability = 0.3666666667
  by: Derangements  (recurrence D(n) = (n-1)(D(n-1) + D(n-2)))
  assumes: 0 <= n <= 170
```

What the worked example means: This calculation determines the number of ways to arrange 5 distinct items such that no item appears in its original position, denoted as 44. It also computes the ratio of these arrangements to all possible permutations, resulting in a probability of 0.366667. As the count of items increases, this probability approaches a specific constant value related to natural logarithms.

Distinct arrangements of the letters of a word — `word_arrangements`

The number of distinct orderings of a word's letters, repeated letters counted once (multiset permutations).

$$\frac{n!}{n_1!\,n_2!\cdots n_r!}$$

symbol		meaning	SI unit
`word`	in	the word (letters A-Z; case is ignored)	—
`arrangements`	out	n! / (n1! n2! ...) over the letter multiplicities	—
`length`	out	number of letters	—
`letter_counts`	out	how often each letter occurs	—

Applies when: letters A-Z only (upper and lower case are the same letter); at most 170 letters.

```
$ discrete-math solve word_arrangements word=MISSISSIPPI
arrangements = 34650
length = 11
letter_counts = {"I":4,"M":1,"P":2,"S":4}
  by: Distinct arrangements of the letters of a word  (product of binomial coefficients, placing
one letter type at a time)
  word = MISSISSIPPI
  assumes: letters A-Z only (upper and lower case are the same letter); at most 170 letters
```

What the worked example means: This calculation determines the number of unique ways to reorder the letters in the input word. It first counts how many times each letter appears, resulting in a total length of 11. Using these frequencies, it computes the multinomial coefficient to find that there are 34650 distinct permutations, treating identical letters as indistinguishable.

Fibonacci numbers — `fibonacci`

F_n with F_0 = 0, F_1 = 1, F_n = F_(n-1) + F_(n-2) (exact integer).

$$F_n = F_{n-1} + F_{n-2},\ F_0 = 0,\ F_1 = 1$$

symbol		meaning	SI unit
`n`	in	the index n (0..1000)	—
`fibonacci`	out	F_n	—

Applies when: 0 <= n <= 1000; F_0 = 0, F_1 = 1.

```
$ discrete-math solve fibonacci n=30
fibonacci = 832040
  by: Fibonacci numbers  (iteration of the recurrence)
  assumes: 0 <= n <= 1000; F_0 = 0, F_1 = 1
```

What the worked example means: This calculation determines the specific term in the Fibonacci sequence identified by the index 30. By applying the recurrence relation where each term is the sum of the two preceding ones, starting from zero and one, the exact integer value for that position is found to be 832040. This result represents the cumulative growth defined by the additive rule up to the requested step.

Integer partitions p(n) — `integer_partitions`

p(n): the number of ways to write n as a sum of positive integers, order ignored.

$$p(n) = \sum_{k \neq 0} (-1)^{k+1} p\left(n - \frac{k(3k-1)}{2}\right)$$

symbol		meaning	SI unit
`n`	in	the integer n (0..2000)	—
`partitions`	out	p(n)	—

Applies when: 0 <= n <= 2000.

```
$ discrete-math solve integer_partitions n=10
partitions = 42
  by: Integer partitions p(n)  (Euler's pentagonal number recurrence)
  assumes: 0 <= n <= 2000
```

What the worked example means: This calculation determines the count of distinct ways to express the integer 10 as a sum of positive integers, ignoring the order of the addends. The result, denoted as 42, represents this total number of partitions. For example, if the input is small, the output reflects the specific combinatorial possibilities available for that value.

Integer solutions of x1 + ... + xk = n (stars and bars) — `stars_and_bars`

How many integer solutions x1 + ... + xk = n have every xi between a lower and an optional upper bound (identical objects into distinct boxes).

$$\#\{x_i \geq 0, \sum x_i = n\} = \binom{n+k-1}{k-1}, \quad \sum_j (-1)^j \binom{k}{j} \binom{n-j(c+1)+k-1}{k-1}$$

symbol		meaning	SI unit
`total`	in	the total n (0..2000)	—
`variables`	in	the number of variables or boxes k (1..50)	—
`lower`	in	lower bound for every variable (0 = non-negative, 1 = positive) (optional)	—
`upper`	in	optional upper bound for every variable (optional)	—
`solutions`	out	number of solutions	—

Applies when: the bounds apply to every variable; a question must say non-negative or positive.

```
$ discrete-math solve stars_and_bars total=10 variables=3 lower=0
solutions = 66
  by: Integer solutions of x1 + ... + xk = n (stars and bars)  (stars and bars after shifting by
the lower bound; inclusion-exclusion over variables exceeding the upper bound)
  assumes: the bounds apply to every variable; a question must say non-negative or positive
```

What the worked example means: There are 66 ways to distribute 10 items among 3 boxes.

Lattice paths in a grid — `lattice_paths`

Monotone lattice paths using unit steps right and up from (0, 0) to (m, n): C(m + n, m).

$$\binom{m+n}{m}$$

symbol		meaning	SI unit
`right`	in	number of steps right m	—
`up`	in	number of steps up n	—
`paths`	out	C(m + n, m)	—

Applies when: m, n >= 0 (the end point is up and to the right); an a by b grid of squares has C(a + b, a) corner-to-corner paths.

```
$ discrete-math solve lattice_paths right=5 up=3
paths = 56
  by: Lattice paths in a grid  (choose which of the m + n steps go right)
  assumes: m, n >= 0 (the end point is up and to the right); an a by b grid of squares has C(a + b,
a) corner-to-corner paths
```

What the worked example means: This calculation determines the total count of unique paths on a grid that start at the bottom-left corner and end at a specific top-right coordinate, moving only one unit right or one unit up at each step. The result, 56, represents the number of ways to arrange 5 rightward moves and 3 upward moves into a single sequence. Essentially, it answers how many distinct routes exist between these two points without backtracking or diagonal movement.

Lucas numbers — `lucas`

L_n with L_0 = 2, L_1 = 1, L_n = L_(n-1) + L_(n-2) (exact integer).

$$L_n = L_{n-1} + L_{n-2},\ L_0 = 2,\ L_1 = 1$$

symbol		meaning	SI unit
`n`	in	the index n (0..1000)	—
`lucas`	out	L_n	—

Applies when: 0 <= n <= 1000; L_0 = 2, L_1 = 1.

```
$ discrete-math solve lucas n=10
lucas = 123
  by: Lucas numbers  (iteration of the recurrence)
  assumes: 0 <= n <= 1000; L_0 = 2, L_1 = 1
```

What the worked example means: This calculation determines the value of the Lucas sequence at a specific position, denoted as n. The sequence begins with two initial values, where the zeroth term is two and the first term is one. Each subsequent term is generated by adding the two preceding terms together, following the same additive recurrence relation used in the Fibonacci sequence but with different starting points. For the input index 10, the resulting integer value is 123. This result represents the exact nth Lucas number, useful in various mathematical identities involving powers of the golden ratio.

Multinomial coefficient — `multinomial`

n! / (k1! k2! ... km!): ways to split n distinct items into labelled groups of the given sizes.

$$\binom{n}{k_1, \ldots, k_m} = \frac{n!}{k_1! \cdots k_m!}$$

symbol		meaning	SI unit
`counts`	in	the group sizes k1, ..., km (whole numbers)	—
`total`	in	the stated total n (optional; must equal the sum of the sizes) (optional)	—
`coefficient`	out	the multinomial coefficient	—
`total`	out	n = k1 + ... + km	—

Applies when: sizes are whole numbers >= 0 adding up to at most 170; a stated total must equal their sum.

```
$ discrete-math solve multinomial "counts=[2, 3, 5]"
coefficient = 2520
total = 10
  by: Multinomial coefficient  (n! divided by each k_i! (exact integers))
  assumes: sizes are whole numbers >= 0 adding up to at most 170; a stated total must equal their
sum
```

What the worked example means: This calculation determines the number of distinct arrangements for a set of items divided into labeled groups. Taking the factorial of the total count and dividing it by the product of the factorials of each group size shows that there are 2520 possible configurations. This result applies when all individual items are considered unique and the groups themselves are distinguishable from one another.

Necklaces (rotations identified) — `necklaces`

Distinct necklaces of n beads in k colours when rotations count as the same (and bracelets, also flips).

$$N(n,k) = \frac{1}{n} \sum_{d \mid n} \varphi(d)\, k^{n/d}$$

symbol		meaning	SI unit
`beads`	in	number of beads n (1..60)	—
`colours`	in	number of colours k (1..60)	—
`necklaces`	out	classes under rotation	—
`bracelets`	out	classes under rotation and reflection	—

Applies when: 1 <= n, k <= 60.

```
$ discrete-math solve necklaces beads=6 colours=2
necklaces = 14
bracelets = 13
  by: Necklaces (rotations identified)  (Burnside's lemma over the cyclic group (and the dihedral
group))
  assumes: 1 <= n, k <= 60
```

What the worked example means: This calculation determines the number of unique circular arrangements using Burnside's lemma to account for rotational symmetry. Given 6 positions and 2 available colors, the method averages the fixed points of all rotations in the cyclic group. The result is 14 classes where any rotation of a pattern is considered identical. Additionally, if reflections are also identified as equivalent, forming bracelets, the count becomes 13. This approach prevents overcounting patterns that look different only due to their starting position on the circle.

Partitions into distinct parts — `partitions_distinct`

q(n): the number of ways to write n as a sum of distinct positive integers (equal to partitions into odd parts).

$$\sum_{n} q(n) x^n = \prod_{k \geq 1} (1 + x^k) = \prod_{k \geq 1} \frac{1}{1 - x^{2k-1}}$$

symbol		meaning	SI unit
`n`	in	the integer n (0..1000)	—
`partitions`	out	q(n)	—

Applies when: 0 <= n <= 1000.

```
$ discrete-math solve partitions_distinct n=10
partitions = 10
  by: Partitions into distinct parts  (0/1 knapsack counting over the parts 1..n)
  assumes: 0 <= n <= 1000
```

What the worked example means: This calculation determines the count of partitions for the integer 10 where each part must be unique. The result, 10, represents the total number of such combinations. This method relies on the equivalence between partitions into distinct parts and partitions into odd parts, effectively counting subsets of integers that sum to the target value.

Partitions into exactly k parts — `partitions_k`

p(n, k): the number of ways to write n as a sum of exactly k positive integers, order ignored.

$$p(n,k) = p(n-1,k-1) + p(n-k,k)$$

symbol		meaning	SI unit
`n`	in	the integer n (0..300)	—
`k`	in	the number of parts (0..300)	—
`partitions`	out	p(n, k)	—

Applies when: 0 <= n, k <= 300.

```
$ discrete-math solve partitions_k n=10 k=3
partitions = 8
  by: Partitions into exactly k parts  (recurrence p(n, k) = p(n-1, k-1) + p(n-k, k))
  assumes: 0 <= n, k <= 300
```

In the worked example, n = 10, k = 3 go through the equation above and give partitions = 8. It applies only when 0 <= n, k <= 300: outside that it says what is wrong instead of returning a number.

Permutations P(n, k) — `permutations`

The number of ordered arrangements of k of n distinct items (exact integer); P(n, n) = n!.

$$P(n,k) = \frac{n!}{(n-k)!}$$

symbol		meaning	SI unit
`n`	in	number of distinct items (0..170)	—
`k`	in	number arranged (0..n)	—
`permutations`	out	P(n, k) = n! / (n-k)!	—

Applies when: 0 <= k <= n <= 170.

```
$ discrete-math solve permutations n=10 k=3
permutations = 720
  by: Permutations P(n, k)  (falling factorial n (n-1) ... (n-k+1))
  assumes: 0 <= k <= n <= 170
```

What the worked example means: This calculation determines the number of ordered arrangements possible when selecting 3 distinct items from a larger set of 10 unique objects. The result, 720, represents every possible sequence in which these specific items can be placed, assuming that order matters and no item is repeated within a single arrangement.

Pigeonhole principle: guaranteed load — `pigeonhole_max`

Placing n objects into k boxes, some box must hold at least ceil(n / k) objects (and this is the best guarantee).

$$\max_i x_i \geq \left\lceil \frac{n}{k} \right\rceil$$

symbol		meaning	SI unit
`items`	in	number of objects n	—
`boxes`	in	number of boxes k	—
`guaranteed`	out	ceil(n / k): the largest load every placement must reach in some box	—

Applies when: n, k >= 1.

```
$ discrete-math solve pigeonhole_max items=25 boxes=7
guaranteed = 4
  by: Pigeonhole principle: guaranteed load  (generalized pigeonhole principle)
  assumes: n, k >= 1
```

What the worked example means: This calculation applies the generalized pigeonhole principle to determine the guaranteed minimum load for any distribution. Dividing the total number of objects, 25, by the number of containers, 7, and rounding up to the nearest integer shows that no matter how the items are arranged, some container will inevitably hold at least 4 items. This value represents the tightest possible lower bound on the maximum occupancy across all valid placements.

Pigeonhole principle: objects needed for a guarantee — `pigeonhole_needed`

How many objects drawn from k categories guarantee at least r of one category: k (r - 1) + 1.

$$N = k(r-1)+1$$

symbol		meaning	SI unit
`boxes`	in	number of categories k	—
`at_least`	in	the guaranteed count r in one category	—
`needed`	out	k (r - 1) + 1	—

Applies when: k, r >= 1.

```
$ discrete-math solve pigeonhole_needed boxes=4 at_least=3
needed = 9
  by: Pigeonhole principle: objects needed for a guarantee  (pigeonhole: k (r - 1) objects can
avoid r in every category)
  assumes: k, r >= 1
```

What the worked example means: This calculation determines the minimum number of objects required to ensure that at least a specific count falls into one category. Taking the number of available categories and multiplying it by the target count reduced by one, then adding a single object gives the threshold where avoidance is no longer possible. The result indicates that with 9 objects distributed among 4 categories, it is impossible for every category to hold fewer than 3 items.

Sequences with repetition (n^k) — `sequences_with_repetition`

The number of length-k sequences over n symbols when symbols may repeat.

$$n^k$$

symbol		meaning	SI unit
`n`	in	number of available symbols (at least 1)	—
`k`	in	length of the sequence	—
`sequences`	out	n^k	—

Applies when: n >= 1; result at most 10^300.

```
$ discrete-math solve sequences_with_repetition n=10 k=4
sequences = 10000
  by: Sequences with repetition (n^k)  (multiplication principle)
  assumes: n >= 1; result at most 10^300
```

What the worked example means: The calculation determines the total count of distinct ordered arrangements, or sequences, that can be formed with a fixed length. By applying the multiplication principle, each of the 4 positions in the sequence is treated as an independent choice from a pool of 10 available symbols. Since repetition is allowed, every position offers the same number of options, leading to a final result of 10000 unique combinations.

Stirling numbers of the first kind — `stirling1`

c(n, k): permutations of n elements with exactly k cycles (unsigned), and the signed s(n, k).

$$c(n,k) = (n-1)\,c(n-1,k) + c(n-1,k-1), \quad x^{\overline{n}} = \sum_{k} c(n,k)\,x^k$$

symbol		meaning	SI unit
`n`	in	number of elements (0..100)	—
`k`	in	number of cycles (0..100)	—
`stirling1`	out	unsigned c(n, k)	—
`signed`	out	signed s(n, k) = (-1)^(n-k) c(n, k)	—

Applies when: 0 <= n, k <= 100.

```
$ discrete-math solve stirling1 n=5 k=2
stirling1 = 50
signed = -50
  by: Stirling numbers of the first kind  (recurrence c(n, k) = (n-1) c(n-1, k) + c(n-1, k-1))
  assumes: 0 <= n, k <= 100
```

What the worked example means: This calculation determines the unsigned Stirling number of the first kind, denoted as c(n,k), which represents the count of distinct permutations of 5 elements that contain exactly 2 disjoint cycles. The result, 50, is derived using the recurrence relation where each step depends on previous values for smaller numbers of elements and cycles. This value indicates how many ways the set can be arranged such that the cycle structure has precisely 2 components.

Stirling numbers of the second kind — `stirling2`

S(n, k): ways to partition an n-element set into k non-empty unlabelled blocks; k! S(n, k) onto functions.

$$S(n,k) = k\,S(n-1,k) + S(n-1,k-1) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^j\binom{k}{j}(k-j)^n$$

symbol		meaning	SI unit
`n`	in	size of the set (0..100)	—
`k`	in	number of blocks (0..100)	—
`stirling2`	out	S(n, k)	—
`surjections`	out	k! S(n, k): onto functions from an n-set to a k-set	—

Applies when: 0 <= n, k <= 100.

```
$ discrete-math solve stirling2 n=7 k=3
stirling2 = 301
surjections = 1806
  by: Stirling numbers of the second kind  (recurrence S(n, k) = k S(n-1, k) + S(n-1, k-1))
  assumes: 0 <= n, k <= 100
```

What the worked example means: This calculation determines the Stirling number of the second kind, which counts the ways to divide a set of 7 distinct items into 3 non-empty, indistinguishable groups. The result, 301, represents this partition count. Additionally, the model computes the number of surjective functions from an n-set to a k-set by multiplying the partition count by the factorial of k, yielding 1806. This relationship highlights that every partition corresponds to multiple labelled mappings depending on how the blocks are assigned to target values.

Cryptography (9)

Affine cipher — `affine_cipher`

Encrypt or decrypt with the affine cipher E(x) = a x + b (mod 26).

$$E(x) = (ax + b) \bmod 26,\; D(y) = a^{-1}(y - b) \bmod 26$$

symbol		meaning	SI unit
`message`	in	the message (letters are kept, everything else is dropped)	—
`a`	in	multiplier (coprime to 26)	—
`b`	in	shift	—
`mode`	in	encrypt or decrypt	—
`result`	out	the result in capital letters, without spaces or punctuation	—

Applies when: gcd(a, 26) = 1; letters A-Z only; other characters are removed; output is upper case.

```
$ discrete-math solve affine_cipher a=5 b=8 "message=AFFINE CIPHER" mode=encrypt
result = IHHWVCSWFRCP
  by: Affine cipher  (E(x) = a x + b, D(y) = a^(-1) (y - b) modulo 26)
  message = AFFINE CIPHER
  mode = encrypt
  assumes: gcd(a, 26) = 1; letters A-Z only; other characters are removed; output is upper case
```

What the worked example means: This calculation applies the affine transformation defined by E(x) = ax + b or its inverse D(y) = a^-1(y - b) modulo 26 to the input message. The system first filters non-letter characters from AFFINE CIPHER and converts the remaining letters into numerical values. Depending on the selected mode, it either encrypts using the parameters 5 and 8, or decrypts using the modular multiplicative inverse of 5. The final numerical results are mapped back to uppercase letters to produce IHHWVCSWFRCP. This process relies on the condition that gcd(a, 26) equals one to ensure the mapping is bijective.

Caesar (shift) cipher — `caesar_cipher`

Encrypt or decrypt a message with a Caesar shift of k letters.

$$E(x) = (x + k) \bmod 26$$

symbol		meaning	SI unit
`message`	in	the message (letters are kept, everything else is dropped)	—
`shift`	in	the shift k	—
`mode`	in	encrypt or decrypt	—
`result`	out	the result in capital letters, without spaces or punctuation	—

Applies when: letters A-Z only are enciphered; other characters are removed; output is upper case.

```
$ discrete-math solve caesar_cipher "message=HELLO WORLD" mode=encrypt shift=3
result = KHOORZRUOG
  by: Caesar (shift) cipher  (each letter x becomes x + k (mod 26); decryption subtracts k)
  message = HELLO WORLD
  mode = encrypt
  assumes: letters A-Z only are enciphered; other characters are removed; output is upper case
```

What the worked example means: This calculation applies a Caesar cipher to the input message by shifting each letter forward or backward in the alphabet based on the specified mode. The final output, KHOORZRUOG, represents the transformed text where every character has been mapped according to the rule E(x) equals x plus k modulo twenty six. Non-letter characters are removed during processing, ensuring the result contains only capital letters.

Diffie-Hellman key exchange — `diffie_hellman`

The public values A = g^a, B = g^b (mod p) and the shared secret g^(ab) mod p.

$$s = B^a \bmod p = A^b \bmod p = g^{ab} \bmod p$$

symbol		meaning	SI unit
`p`	in	the prime modulus	—
`g`	in	the generator (1 < g < p)	—
`a`	in	Alice's secret exponent	—
`b`	in	Bob's secret exponent	—
`A`	out	Alice's public value g^a mod p	—
`B`	out	Bob's public value g^b mod p	—
`shared`	out	shared secret g^(ab) mod p	—

Applies when: p prime; 1 < g < p.

```
$ discrete-math solve diffie_hellman p=23 g=5 a=6 b=15
A = 8
B = 19
shared = 2
  by: Diffie-Hellman key exchange  (modular exponentiation: B^a = A^b = g^(ab))
  assumes: p prime; 1 < g < p
```

What the worked example means: This calculation determines the Diffie-Hellman shared secret by performing modular exponentiation. Alice computes her public value as 8 using her secret exponent, while Bob computes his public value as 19. The final shared secret 2 is obtained by raising each party's public value to the other's private exponent modulo the prime modulus. This result represents a symmetric key that both parties can use for secure communication without ever transmitting the secret itself over the channel.

ElGamal encryption — `elgamal_encrypt`

ElGamal ciphertext (c1, c2) = (g^k, m h^k) mod p with public key h = g^x.

$$c_1 = g^k,\ c_2 = m\,h^k \pmod{p}, \quad m = c_2\,c_1^{-x}$$

symbol		meaning	SI unit
`p`	in	the prime modulus	—
`g`	in	the generator	—
`x`	in	the private key	—
`k`	in	the ephemeral exponent	—
`m`	in	the message (1 <= m < p)	—
`h`	out	public key g^x mod p	—
`c1`	out	g^k mod p	—
`c2`	out	m h^k mod p	—

Applies when: p prime; 1 < g < p; 1 <= m < p.

```
$ discrete-math solve elgamal_encrypt p=23 g=5 x=6 k=3 m=10
h = 8
c1 = 10
c2 = 14
  by: ElGamal encryption  (modular exponentiation)
  assumes: p prime; 1 < g < p; 1 <= m < p
```

What the worked example means: ElGamal encryption generates a public key derived from the private key as 8, then produces two ciphertext values: 10 representing the ephemeral component and 14 containing the masked message. The result ensures that the original message remains hidden under modular arithmetic assumptions while allowing decryption using the private key.

RSA decryption — `rsa_decrypt`

The RSA plaintext m = c^d mod n of a ciphertext c.

$$m = c^{d} \bmod n$$

symbol		meaning	SI unit
`c`	in	the ciphertext (0 <= c < n)	—
`d`	in	private exponent	—
`n`	in	modulus	—
`m`	out	plaintext c^d mod n	—

Applies when: 0 <= c < n.

```
$ discrete-math solve rsa_decrypt c=2790 d=2753 n=3233
m = 65
  by: RSA decryption  (modular exponentiation)
  assumes: 0 <= c < n
```

What the worked example means: This calculation determines the original message by applying modular exponentiation to the received data. It takes the ciphertext 2790, raises it to the power of the private key 2753, and reduces the result modulo the public modulus 3233. The final value, 65, represents the recovered plaintext number that corresponds to the encrypted input before any padding was applied.

RSA decryption from p, q and e (CRT) — `rsa_decrypt_pq`

Decrypt an RSA ciphertext knowing the primes: compute d, then m by the Chinese remainder theorem.

$$m = m_q + q\left(q^{-1}(m_p - m_q) \bmod p\right)$$

symbol		meaning	SI unit
`c`	in	the ciphertext (0 <= c < n)	—
`p`	in	first prime	—
`q`	in	second prime	—
`e`	in	public exponent	—

symbol		meaning	SI unit
`m`	out	the plaintext	—
`d`	out	private exponent e^(-1) mod phi(n)	—
`n`	out	modulus p q	—

Applies when: p, q distinct primes; gcd(e, phi(n)) = 1; 0 <= c < n.

```
$ discrete-math solve rsa_decrypt_pq c=2790 p=61 q=53 e=17
m = 65
d = 2753
n = 3233
  by: RSA decryption from p, q and e (CRT)  (d = e^(-1) mod phi; m_p = c^(d mod p-1) mod p, m_q
likewise; Garner's recombination)
  assumes: p, q distinct primes; gcd(e, phi(n)) = 1; 0 <= c < n
```

What the worked example means: This calculation determines the original message from the ciphertext using RSA decryption optimized by the Chinese Remainder Theorem. It first computes the modulus n as 3233 and the private exponent d as 2753, derived from the public exponent e and the primes p and q. Then, it calculates intermediate values modulo p and q separately before recombining them to yield the final plaintext m, which equals 65. This method is significantly faster than direct modular exponentiation because it operates on smaller numbers.

RSA encryption — `rsa_encrypt`

The RSA ciphertext c = m^e mod n of a message number m.

$$c = m^e \bmod n$$

symbol		meaning	SI unit
`m`	in	the message as a number (0 <= m < n)	—
`e`	in	public exponent	—
`n`	in	modulus	—
`c`	out	ciphertext m^e mod n	—

Applies when: 0 <= m < n.

```
$ discrete-math solve rsa_encrypt m=65 e=17 n=3233
c = 2790
  by: RSA encryption  (modular exponentiation)
  assumes: 0 <= m < n
```

What the worked example means: This calculation determines the RSA ciphertext 2790 by applying modular exponentiation to the numeric message 65 using the public exponent 17 and the modulus 3233. The result represents the encrypted form of the original data, which can only be recovered by a party possessing the corresponding private key. This process relies on the mathematical property that computing large powers modulo a composite number is easy for the sender but computationally infeasible to reverse without specific knowledge of the factors of the modulus.

RSA key from p, q and e — `rsa_keygen`

The RSA modulus n = p q, phi(n) = (p-1)(q-1) and the private exponent d = e^(-1) mod phi(n).

$$n = pq,\ \varphi = (p-1)(q-1),\ d \equiv e^{-1} \pmod{\varphi}$$

symbol		meaning	SI unit
`p`	in	first prime	—
`q`	in	second prime	—
`e`	in	public exponent	—
`n`	out	modulus p q	—
`phi`	out	Euler's phi(n) = (p-1)(q-1)	—
`d`	out	private exponent e^(-1) mod phi(n)	—

Applies when: p, q distinct primes; 1 < e < phi(n) and gcd(e, phi(n)) = 1; d is taken modulo phi(n) (not lambda(n)).

```
$ discrete-math solve rsa_keygen p=61 q=53 e=17
n = 3233
phi = 3120
d = 2753
  by: RSA key from p, q and e  (extended Euclidean algorithm for the inverse of e modulo phi(n))
  assumes: p, q distinct primes; 1 < e < phi(n) and gcd(e, phi(n)) = 1; d is taken modulo phi(n)
(not lambda(n))
```

What the worked example means: This calculation derives the RSA key components from the two distinct primes 61 and 53 along with the public exponent 17. It first computes the modulus n as the product of the primes, resulting in 3233. Next, it determines Euler's totient function phi using the formula involving the predecessors of the primes, yielding 3120. Finally, it finds the modular multiplicative inverse of the public exponent modulo the totient to obtain the private exponent d, which equals 2753. These values collectively form the necessary parameters for secure encryption and decryption operations.

Vigenère cipher — `vigenere_cipher`

Encrypt or decrypt with the Vigenère cipher under a keyword.

$$c_j = (m_j + k_{j \bmod |k|}) \bmod 26$$

symbol		meaning	SI unit
`message`	in	the message (letters are kept, everything else is dropped)	—
`key`	in	the keyword (letters)	—
`mode`	in	encrypt or decrypt	—
`result`	out	the result in capital letters, without spaces or punctuation	—

Applies when: letters A-Z only; other characters are removed first, so the key runs over letters only; output is upper case.

```
$ discrete-math solve vigenere_cipher key=LEMON "message=ATTACK AT DAWN" mode=encrypt
result = LXFOPVEFRNHR
  by: Vigenère cipher  (add (encrypt) or subtract (decrypt) the repeating key letters modulo 26)
  key = LEMON
  message = ATTACK AT DAWN
  mode = encrypt
  assumes: letters A-Z only; other characters are removed first, so the key runs over letters only;
output is upper case
```

What the worked example means: This calculation applies the Vigenère cipher to the input message ATTACK AT DAWN using the keyword LEMON. The algorithm processes only alphabetic characters, converting them to uppercase and discarding any non-letter symbols. Depending on the selected encrypt, it either adds or subtracts the repeating key values modulo twenty-six to produce the encrypted or decrypted text LXFOPVEFRNHR.

Graph theory (25)

Articulation points (cut vertices) — `articulation_points`

The vertices whose removal increases the number of connected components.

$$u \text{ cut vertex} \iff \exists \text{ child } v : \ \text{low}(v) \geq \text{disc}(u)$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`count`	out	number of articulation points	—
`points`	out	the articulation points, ascending	—

Applies when: undirected simple graph, unweighted.

```
$ discrete-math solve articulation_points "edges=A-B, B-C, C-A, C-D, D-E"
count = 2
points = ["C","D"]
  by: Articulation points (cut vertices)  (Tarjan's depth-first lowlink algorithm)
  assumes: undirected simple graph, unweighted
```

What the worked example means: This calculation determines the vertices whose removal would increase the number of connected components in the undirected graph defined by A-B, B-C, C-A, C-D, D-E. The result shows that there are 2 such critical nodes, identified as [C, D]. These points represent single points of failure in the network structure, meaning their absence fragments the connectivity of the system.

Bipartiteness test — `bipartite`

Whether the vertices of an undirected graph split into two sets with every edge between them (no odd cycle).

$$G \text{ bipartite} \iff G \text{ has no odd cycle}$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`is_bipartite`	out	true when the graph is 2-colourable	—

Applies when: undirected simple graph, unweighted.

```
$ discrete-math solve bipartite "edges=A-B, B-C, C-D, D-A"
is_bipartite = true
  by: Bipartiteness test  (2-colouring by graph search; a conflict exposes an odd cycle)
  assumes: undirected simple graph, unweighted
```

What the worked example means: This calculation determines whether the vertices of an undirected simple graph can be divided into two disjoint sets such that every edge connects a vertex from one set to the other. By attempting to color the vertices using two colors via graph search, the algorithm checks for conflicts that indicate the presence of an odd cycle. The result yes is true if such a coloring is possible without conflict, confirming the graph is bipartite based on the input structure defined by A-B, B-C, C-D, D-A.

Breadth-first search order — `bfs_order`

The order in which breadth-first search from a source visits the vertices, and each vertex's distance in edges.

$$d(v) = d(u) + 1 \text{ when } v \text{ is first seen from } u$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`source`	in	the start vertex	—
`directed`	in	true for a directed graph (edges u->v) (optional)	—
`order`	out	visiting order	—
`reached`	out	number of vertices reached	—
`distances`	out	number of edges from the source to each reached vertex	—

Applies when: simple unweighted graph; TIE RULE: neighbours are visited in ascending order of their names (numerically when the names are integers).

```
$ discrete-math solve bfs_order "edges=A-B, A-C, B-D, C-D, D-E" source=A
order = ["A","B","C","D","E"]
reached = 5
distances = {"A":0,"B":1,"C":1,"D":2,"E":3}
  by: Breadth-first search order  (queue-based breadth-first search)
  source = A
  assumes: simple unweighted graph; TIE RULE: neighbours are visited in ascending order of their
names (numerically when the names are integers)
```

What the worked example means: This calculation determines the traversal sequence and shortest path edge counts for a simple unweighted graph using queue-based breadth-first search. Starting at vertex A, the algorithm explores neighbors in ascending name order, producing the visitation list [A, B, C, D, E]. The result shows that 5 vertices are reachable, with their respective distances from the source given by {A: 0, B: 1, C: 1, D: 2, E: 3}, where each value represents the minimum number of edges required to reach that specific vertex.

Bridges (cut edges) — `bridges`

The edges whose removal increases the number of connected components.

$$(u, v) \text{ bridge} \iff \text{low}(v) > \text{disc}(u)$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`count`	out	number of bridges	—
`bridges`	out	the bridges [u, v] with u < v, sorted	—

Applies when: undirected simple graph, unweighted.

```
$ discrete-math solve bridges "edges=A-B, B-C, C-A, C-D, D-E"
count = 2
bridges = [["C","D"],["D","E"]]
  by: Bridges (cut edges)  (Tarjan's depth-first lowlink algorithm)
  assumes: undirected simple graph, unweighted
```

What the worked example means: This calculation determines the number of critical edges whose removal would disconnect the graph. By applying a depth-first search algorithm to the input edges A-B, B-C, C-A, C-D, D-E, it identifies specific connections where the lowest discovery time reachable from a subtree is strictly greater than the discovery time of its parent node. The result shows there are 2 such bridges, which are reported as pairs of vertices in [[C, D], [D, E]]. These edges represent single points of failure, meaning their absence breaks connectivity between otherwise linked components.

Chromatic number of a small graph — `chromatic_number`

The fewest colours needed so that adjacent vertices differ, with one optimal colouring.

$$\chi(G) = \min\{k : G \text{ is } k\text{-colourable}\}$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`chromatic_number`	out	chi(G)	—
`colouring`	out	an optimal colouring (colours 1..chi)	—

Applies when: undirected simple graph, unweighted, at most 12 vertices; the colouring is the first one found trying vertices in ascending name order, colours 1, 2, ...

```
$ discrete-math solve chromatic_number "edges=A-B, B-C, C-A, C-D"
chromatic_number = 3
colouring = {"A":1,"B":2,"C":3,"D":1}
  by: Chromatic number of a small graph  (exact backtracking for k = 1, 2, ... colours)
  assumes: undirected simple graph, unweighted, at most 12 vertices; the colouring is the first one
found trying vertices in ascending name order, colours 1, 2, ...
```

What the worked example means: This calculation determines the minimum number of colors required to color the vertices of an undirected simple graph such that no two adjacent vertices share the same color. Using exact backtracking, it tests increasing numbers of colors until a valid assignment is found. The result shows that the graph requires 3 colors, and provides one specific optimal coloring scheme where each vertex is assigned a value from {A: 1, B: 2, C: 3, D: 1}. This metric is essential for understanding constraints in scheduling and resource allocation problems.

Clique number — `clique_number`

The size of a largest complete subgraph, with one maximum clique.

$$\omega(G) = \max\{|K| : K \text{ complete}\}$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`clique_number`	out	omega(G)	—
`clique`	out	a maximum clique (the lexicographically smallest), ascending	—

Applies when: undirected simple graph, unweighted, at most 60 vertices.

```
$ discrete-math solve clique_number "edges=A-B, B-C, C-A, C-D, B-D, A-D, D-E"
clique_number = 4
clique = ["A","B","C","D"]
  by: Clique number  (Bron-Kerbosch with pivoting)
  assumes: undirected simple graph, unweighted, at most 60 vertices
```

What the worked example means: This calculation determines the size of the largest complete subgraph within the provided network. Using the Bron-Kerbosch algorithm with pivoting, it identifies a set of vertices where every member is directly connected to all others in that set. The result shows that the clique number is 4, meaning the largest such group contains this many nodes. The specific vertices forming this maximum clique are listed as [A, B, C, D]. This metric is crucial for understanding tightly-knit communities or fully interconnected components in undirected simple graphs.

Connected components — `connected_components`

The connected components of an undirected graph: how many, their sizes and members, and whether it is connected.

$$V = C_1 \sqcup \cdots \sqcup C_c$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`count`	out	number of connected components	—
`sizes`	out	component sizes, descending	—
`is_connected`	out	true when there is one component	—
`components`	out	members of each component (ascending), components ordered by their first member	—

Applies when: undirected simple graph, unweighted; the vertices are those named in the edges.

```
$ discrete-math solve connected_components "edges=A-B, C-D, D-E"
count = 2
sizes = [3,2]
is_connected = false
components = [["A","B"],["C","D","E"]]
  by: Connected components  (graph search from each unvisited vertex)
  assumes: undirected simple graph, unweighted; the vertices are those named in the edges
```

What the worked example means: This calculation determines the connected components of an undirected graph by performing a graph search from each unvisited vertex. The result identifies how many separate groups of vertices exist, listed as 2, along with their respective member counts in descending order, shown as [3, 2]. It also specifies whether the entire graph forms a single connected unit, indicated by no, and lists the specific vertices belonging to each component, ordered by their first member.

Cycle detection in a directed graph — `directed_cycle`

Whether a directed graph contains a directed cycle (equivalently, whether it is a DAG).

$$G \text{ acyclic} \iff \text{a topological order exists}$$

symbol		meaning	SI unit
`edges`	in	the directed graph as edges u->v	—
`has_cycle`	out	true when a directed cycle exists	—
`is_dag`	out	true when the graph is acyclic	—

Applies when: directed simple graph, unweighted.

```
$ discrete-math solve directed_cycle "edges=A-B, B-C, C-A"
has_cycle = true
is_dag = false
  by: Cycle detection in a directed graph  (Kahn's algorithm: the graph is acyclic exactly when
every vertex is removed)
  assumes: directed simple graph, unweighted
```

What the worked example means: This calculation applies Kahn's algorithm to the input edges A-B, B-C, C-A to determine if the directed graph contains any cycles. The method checks if a topological order exists, which is equivalent to verifying if every vertex can be removed by processing nodes with zero incoming edges. The resulting boolean values yes and no confirm whether the structure is acyclic or contains at least one directed loop.

Cycles in an undirected graph (cyclomatic number) — `cycle_rank`

Whether an undirected graph has a cycle, and its cyclomatic number m - n + c (independent cycles).

$$\mu(G) = |E| - |V| + c(G)$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`cyclomatic_number`	out	m - n + c: the number of independent cycles	—
`has_cycle`	out	true when the graph contains a cycle	—

Applies when: undirected simple graph, unweighted.

```
$ discrete-math solve cycle_rank "edges=A-B, B-C, C-A, D-E"
cyclomatic_number = 1
has_cycle = true
  by: Cycles in an undirected graph (cyclomatic number)  (count components; every edge beyond a
spanning forest closes one independent cycle)
  assumes: undirected simple graph, unweighted
```

What the worked example means: This calculation determines the cyclomatic number, which represents the count of independent cycles in the undirected graph. It is derived from the total edges and vertices, adjusted by the number of connected components. The result, 1, quantifies the redundancy or complexity of the network structure. A value greater than zero confirms that at least one cycle exists within the graph.

Depth-first search order — `dfs_order`

The preorder in which depth-first search from a source visits the vertices.

$$\mathbf{visit}(u):\ \mathbf{mark}\ u;\ \forall v \in N(u)\ \mathbf{ascending, unmarked:}\ \mathbf{visit}(v)$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`source`	in	the start vertex	—
`directed`	in	true for a directed graph (edges u->v) (optional)	—
`order`	out	visiting order (preorder)	—
`reached`	out	number of vertices reached	—

Applies when: simple unweighted graph; TIE RULE: recursive DFS that tries neighbours in ascending order of their names (numerically when the names are integers).

```
$ discrete-math solve dfs_order "edges=A-B, A-C, B-D, C-D, D-E" source=A
order = ["A","B","D","C","E"]
reached = 5
  by: Depth-first search order  (depth-first search (recursive semantics, kept on an explicit stack
of neighbour iterators))
  source = A
  assumes: simple unweighted graph; TIE RULE: recursive DFS that tries neighbours in ascending
order of their names (numerically when the names are integers)
```

What the worked example means: The depth-first search visiting order is [A, B, D, C, E] and the number of reached vertices is 5.

Diameter, radius and center — `graph_diameter`

Eccentricities of a connected unweighted graph: the diameter (largest), radius (smallest), center and periphery.

$$\operatorname{diam} G = \max_v e(v),\ \operatorname{rad} G = \min_v e(v)$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`diameter`	out	largest distance between two vertices	—
`radius`	out	smallest eccentricity	—
`center`	out	vertices of smallest eccentricity (ascending)	—
`periphery`	out	vertices of largest eccentricity (ascending)	—

Applies when: connected undirected simple graph, unweighted (distances count edges).

```
$ discrete-math solve graph_diameter "edges=A-B, B-C, C-D, D-E, B-F"
diameter = 4
radius = 2
center = ["C"]
periphery = ["A","E","F"]
  by: Diameter, radius and center  (breadth-first search from every vertex)
  assumes: connected undirected simple graph, unweighted (distances count edges)
```

What the worked example means: Using breadth-first search from every vertex gives the eccentricity for each node in this connected unweighted graph. The diameter, which is the largest distance between any two vertices, is found to be 4. The radius, representing the smallest eccentricity among all vertices, is 2. The center consists of the vertices with this minimum eccentricity, listed as [C], while the periphery includes those with the maximum eccentricity, listed as [A, E, F]. These values indicate how spread out the network is and identify the most central and peripheral nodes.

Eulerian circuits and paths — `eulerian`

Whether an undirected graph has an Euler circuit (closed trail using every edge once) or an Euler path.

$$\text{circuit} \iff \text{connected} \wedge \forall v:\ 2 \mid \deg v$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`odd_vertices`	out	number of vertices of odd degree	—
`connected`	out	true when the graph is connected	—
`has_circuit`	out	Euler circuit exists: connected and no odd vertex	—
`has_path`	out	Euler path exists: connected and 0 or 2 odd vertices	—

Applies when: undirected simple graph, unweighted; the vertices are those named in the edges.

```
$ discrete-math solve eulerian "edges=A-B, B-C, C-A, C-D, D-E, E-C"
odd_vertices = 0
connected = true
has_circuit = true
has_path = true
  by: Eulerian circuits and paths  (Euler's theorem: degrees and connectivity)
  assumes: undirected simple graph, unweighted; the vertices are those named in the edges
```

What the worked example means: This calculation determines if an undirected graph allows a trail that uses every edge exactly once. It counts the number of vertices with an odd degree, which is currently 0, and checks if all edges form a single connected component. An Euler circuit exists only if the graph is connected and every vertex has an even degree, while an Euler path exists if it is connected and has either zero or two odd-degree vertices.

Girth (shortest cycle length) — `girth`

The length of a shortest cycle of an undirected graph (none for a forest).

$$g(G) = \min\{|C| : C \text{ a cycle of } G\}$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`girth`	out	length of the shortest cycle (none when the graph is acyclic)	—

Applies when: undirected simple graph, unweighted.

```
$ discrete-math solve girth "edges=A-B, B-C, C-D, D-A, A-E, E-F, F-A"
girth = 3
  by: Girth (shortest cycle length)  (breadth-first search from every vertex; a non-tree edge (u,
v) closes a cycle of length d(u) + d(v) + 1)
  assumes: undirected simple graph, unweighted
```

What the worked example means: This calculation determines the girth, which is the length of the shortest cycle in an undirected simple graph. By performing breadth-first searches from every vertex and identifying non-tree edges that close cycles, the algorithm finds the minimum cycle length. The resulting value 3 indicates how tightly connected the graph's local structure is, with smaller values implying shorter loops among nodes.

Graphical degree sequence — `graphical_sequence`

Whether a list of non-negative integers is the degree sequence of some simple graph.

$$\sum_{i \le k} d_i \le k(k-1) + \sum_{i > k} \min(d_i, k) \ \ \forall k, \quad \sum d_i \text{ even}$$

symbol		meaning	SI unit
`degrees`	in	the proposed degrees (whole numbers >= 0)	—
`is_graphical`	out	true when a simple graph has exactly these degrees	—

Applies when: whole numbers >= 0.

```
$ discrete-math solve graphical_sequence "degrees=[3, 3, 2, 2, 2]"
is_graphical = true
  by: Graphical degree sequence  (Havel-Hakimi reduction)
  assumes: whole numbers >= 0
```

What the worked example means: This calculation applies the Havel-Hakimi reduction to the input list of degrees [3, 3, 2, 2, 2] to verify if a simple graph can exist with these specific vertex connections. The process checks two main conditions: that the total sum of degrees is even and that the Erdős-Gallai inequalities hold for all subsets of vertices. If both conditions are satisfied, the output yes indicates that such a graph is possible; otherwise, it signifies that no simple graph can realize this degree sequence.

Hamiltonian cycle of a small graph — `hamiltonian_cycle`

Whether an undirected graph has a cycle through every vertex exactly once, with one such cycle.

$$v_1 v_2 \cdots v_n v_1,\ \{v_i\} = V$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`exists`	out	true when a Hamiltonian cycle exists	—
`cycle`	out	a Hamiltonian cycle starting and ending at the smallest vertex (none if it does not exist)	—

Applies when: undirected simple graph, unweighted, at most 12 vertices; the cycle shown is the first found extending through neighbours in ascending order.

```
$ discrete-math solve hamiltonian_cycle "edges=A-B, B-C, C-D, D-A, A-C"
exists = true
cycle = ["A","B","C","D","A"]
  by: Hamiltonian cycle of a small graph  (backtracking search from the smallest vertex)
  assumes: undirected simple graph, unweighted, at most 12 vertices; the cycle shown is the first
found extending through neighbours in ascending order
```

What the worked example means: The model determines if an undirected graph possesses a cycle visiting every vertex exactly once. Using backtracking search from the smallest vertex, it analyzes the input edges A-B, B-C, C-D, D-A, A-C. If successful, it outputs a specific path like [A, B, C, D, A] and sets the existence flag to yes, indicating the graph allows full traversal without repetition.

Independence number — `independence_number`

The size of a largest set of pairwise non-adjacent vertices, with one maximum independent set.

$$\alpha(G) = \omega(\overline{G})$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`independence_number`	out	alpha(G)	—

symbol		meaning	SI unit
`independent_set`	out	a maximum independent set (the lexicographically smallest), ascending	—

Applies when: undirected simple graph, unweighted, at most 60 vertices; the vertices are those named in the edges.

```
$ discrete-math solve independence_number "edges=1-2, 2-3, 3-4, 4-5, 5-1"
independence_number = 2
independent_set = ["1","3"]
  by: Independence number  (maximum clique of the complement graph (Bron-Kerbosch))
  assumes: undirected simple graph, unweighted, at most 60 vertices; the vertices are those named
in the edges
```

What the worked example means: This calculation finds the largest group of vertices in a graph that are not connected to each other. The result, 2, represents the size of this maximum independent set, while [1, 3] lists the specific vertices involved. This information helps identify resources or tasks that can operate simultaneously without conflict.

Number of spanning trees (Kirchhoff) — `spanning_tree_count`

How many spanning trees an undirected simple graph has (0 if it is disconnected).

$$\tau(G) = \det L_{11}, \quad L = D - A$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`spanning_trees`	out	number of spanning trees tau(G)	—

Applies when: undirected simple graph, unweighted, at most 60 vertices.

```
$ discrete-math solve spanning_tree_count "edges=A-B, B-C, C-D, D-A, A-C"
spanning_trees = 8
  by: Number of spanning trees (Kirchhoff)  (Kirchhoff's matrix-tree theorem: any cofactor of the
Laplacian, by exact rational elimination)
  assumes: undirected simple graph, unweighted, at most 60 vertices
```

What the worked example means: Using Kirchhoff's matrix-tree theorem gives the number of spanning trees for the input graph defined by A-B, B-C, C-D, D-A, A-C. The result is 8, which represents the total count of distinct subgraphs that include every vertex exactly once without forming any cycles. This value indicates how many different ways the network can be fully connected while minimizing redundancy.

Number of triangles — `graph_triangles`

How many triangles (3-cycles) an undirected graph contains.

$$t(G) = \tfrac{1}{6}\operatorname{tr}(A^3)$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`triangles`	out	number of triangles	—

Applies when: undirected simple graph, unweighted.

```
$ discrete-math solve graph_triangles "edges=A-B, B-C, C-A, C-D, D-A"
triangles = 2
  by: Number of triangles  (for each edge, count common neighbours; every triangle is seen from its
three edges)
  assumes: undirected simple graph, unweighted
```

What the worked example means: This calculation determines the total count of three-cycles, or triangles, present in an undirected simple graph. The result, 2, is derived by analyzing the connections between nodes to identify sets of three vertices that are mutually linked. This metric provides insight into the local clustering structure of the network.

Number of walks of a given length — `walks_count`

How many walks of exactly k edges lead from u to v (the (u, v) entry of A^k).

$$\#\mathbf{walks}_k(u, v) = (A^k)_{uv}$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`source`	in	start vertex u	—
`target`	in	end vertex v	—
`length`	in	the walk length k (0..200)	—
`directed`	in	true for a directed graph (edges u->v) (optional)	—
`walks`	out	number of walks of length k from u to v	—

Applies when: simple unweighted graph; vertices may repeat along a walk.

Run `discrete-math info walks_count` for its inputs, then `discrete-math solve walks_count name=value ...`.

What the worked example means: This calculation determines the total count of distinct walks that start at vertex A, end at vertex B, and consist of exactly 3 edges. Applying the adjacency matrix method shows that there are 3 such sequences. Unlike paths, these walks allow vertices and edges to be visited multiple times, making this a measure of connectivity strength over specific step counts in the graph defined by A-B, B-C, C-A.

Prüfer code of a labelled tree — `prufer_encode`

The Prüfer sequence (length n - 2) of a tree on vertices labelled 1..n.

$$(a_1, \ldots, a_{n-2}),\ a_i = \text{neighbour of the smallest leaf}$$

symbol		meaning	SI unit
`edges`	in	the tree's edges, vertices numbered 1..n (or 0..n-1)	—
`sequence`	out	the Prüfer sequence	—
`vertices`	out	number of vertices n	—

Applies when: a tree whose vertices are numbered 1..n or 0..n-1; the smallest leaf is removed at every step.

```
$ discrete-math solve prufer_encode "edges=1-4, 2-4, 3-4, 4-5, 5-6"
sequence = [4,4,4,5]
vertices = 6
  by: Prüfer code of a labelled tree  (repeatedly remove the smallest leaf and record its
neighbour)
  assumes: a tree whose vertices are numbered 1..n or 0..n-1; the smallest leaf is removed at every
step
```

What the worked example means: This calculation determines the Prüfer sequence, a unique code representing a labelled tree. Starting from the edges provided as 1-4, 2-4, 3-4, 4-5, 5-6, the algorithm iteratively removes the smallest leaf node and records its neighbor until only two nodes remain. The resulting sequence, shown here as [4, 4, 4, 5], has a length of n minus two, where n is the total vertex count 6. This encoding allows for compact storage and reconstruction of the original tree structure.

Strongly connected components — `strongly_connected`

The strongly connected components of a directed graph (vertices mutually reachable).

$$u \sim v \iff u \rightsquigarrow v \wedge v \rightsquigarrow u$$

symbol		meaning	SI unit
`edges`	in	the directed graph as edges u->v	—
`count`	out	number of strongly connected components	—
`sizes`	out	component sizes, descending	—
`components`	out	members of each component (ascending), ordered by first member	—

Applies when: directed simple graph, unweighted.

```
$ discrete-math solve strongly_connected "edges=A-B, B-C, C-A, C-D"
count = 2
sizes = [3,1]
components = [["A","B","C"],["D"]]
  by: Strongly connected components  (Tarjan's algorithm)
  assumes: directed simple graph, unweighted
```

What the worked example means: Tarjan's algorithm partitions the directed graph defined by edges A-B, B-C, C-A, C-D into maximal subsets where every vertex is mutually reachable from every other vertex. The result identifies 2 distinct strongly connected components, with their respective member sets listed in [[A, B, C], [D]] and their cardinalities sorted in descending order as [3, 1]. This decomposition reveals the core structural units of dependency within the network.

Topological sort — `topological_sort`

An ordering of a directed acyclic graph's vertices in which every edge u->v has u before v.

$$(u \to v) \in E \Rightarrow \pi(u) < \pi(v)$$

symbol		meaning	SI unit
`edges`	in	the directed graph as edges u->v	—
`order`	out	the lexicographically smallest topological order	—
`vertices`	out	number of vertices	—

Applies when: directed acyclic simple graph, unweighted; TIE RULE: among the vertices free at each step the smallest name comes first (numerically when integers): the lexicographically smallest order.

```
$ discrete-math solve topological_sort "edges=A-C, B-C, C-D, B-D"
order = ["A","B","C","D"]
vertices = 4
  by: Topological sort  (Kahn's algorithm with a min-priority queue of in-degree-zero vertices)
  assumes: directed acyclic simple graph, unweighted; TIE RULE: among the vertices free at each
step the smallest name comes first (numerically when integers): the lexicographically smallest
order
```

What the worked example means: This calculation determines the lexicographically smallest topological ordering of the input directed acyclic graph. Using Kahn's algorithm with a min-priority queue, it processes vertices with zero in-degree by selecting the smallest available label at each step to ensure uniqueness. The resulting sequence [A, B, C, D] guarantees that for every edge from one vertex to another, the source appears before the destination, satisfying all dependency constraints while maintaining the required alphabetical or numerical tie-breaking rule across the total count of 4 vertices.

Tree from a Prüfer code — `prufer_decode`

The labelled tree on 1..n (n = length + 2) whose Prüfer sequence is given.

$$\deg(v) = 1 + \#\{i : a_i = v\}$$

symbol		meaning	SI unit
`sequence`	in	the Prüfer sequence (entries between 1 and n)	—
`edges`	out	the tree's edges [u, v] with u < v, sorted	—
`vertices`	out	number of vertices n	—

Applies when: vertices are numbered 1..n with n = length + 2.

```
$ discrete-math solve prufer_decode "sequence=[4, 4, 4, 5]"
edges = [[1,4],[2,4],[3,4],[4,5],[5,6]]
vertices = 6
  by: Tree from a Prüfer code  (attach each entry to the smallest current leaf; join the last two
leaves)
  assumes: vertices are numbered 1..n with n = length + 2
```

What the worked example means: This algorithm reconstructs a labelled tree from its Prüfer sequence. Given an input sequence, it determines the number of vertices as [4, 4, 4, 5] plus two, resulting in 6 vertices. The method attaches each entry to the smallest current leaf and joins the last two leaves to produce the edge list [[1, 4], [2, 4], [3, 4], [4, 5], [5, 6]]. This process establishes the bijection between sequences and trees, confirming that every valid sequence corresponds to exactly one unique tree structure.

Tree test — `is_tree`

Whether an undirected graph is a tree: connected with exactly n - 1 edges (equivalently, connected and acyclic).

$$T \text{ tree} \iff T \text{ connected} \wedge |E| = |V| - 1$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`is_tree`	out	true when the graph is a tree	—
`vertices`	out	number of vertices	—
`edges`	out	number of edges	—
`connected`	out	true when the graph is connected	—

Applies when: undirected simple graph, unweighted; the vertices are those named in the edges.

```
$ discrete-math solve is_tree "edges=1-2, 1-3, 3-4, 3-5"
is_tree = true
vertices = 5
edges = 4
connected = true
  by: Tree test  (count edges and test connectivity)
  assumes: undirected simple graph, unweighted; the vertices are those named in the edges
```

What the worked example means: This calculation determines whether an undirected graph is a tree by examining its structure. It counts the number of distinct vertices involved in the edge list, resulting in 5, and counts the total number of edges, yielding 4. The method checks if the graph is connected, represented by yes, and verifies if the edge count equals the vertex count minus one. If both conditions are met, the output yes is true, confirming the graph is a tree; otherwise, it is false.

Vertex degrees and degree sequence — `graph_degrees`

The degree of every vertex of an undirected simple graph, its degree sequence, maximum and minimum degree.

$$\sum_{v} \deg v = 2|E|$$

symbol		meaning	SI unit
`edges`	in	the graph as a list of edges u-v (an optional third entry is the weight)	—
`vertices`	out	number of vertices (those named in the edges)	—

symbol		meaning	SI unit
`edges`	out	number of edges	—
`degrees`	out	degree of each vertex	—
`degree_sequence`	out	degrees in descending order	—
`max_degree`	out	maximum degree	—
`min_degree`	out	minimum degree	—

Applies when: undirected simple graph (no loops, no repeated edges), unweighted; the vertices are those named in the edges.

```
$ discrete-math solve graph_degrees "edges=A-B, A-C, B-C, C-D"
vertices = 4
edges = 4
degrees = {"A":2,"B":2,"C":3,"D":1}
degree_sequence = [3,2,2,1]
max_degree = 3
min_degree = 1
  by: Vertex degrees and degree sequence  (count the edges at each vertex (handshake lemma: the
degrees add up to twice the number of edges))
  assumes: undirected simple graph (no loops, no repeated edges), unweighted; the vertices are
those named in the edges
```

What the worked example means: This calculation determines the degree of each vertex in an undirected simple graph by counting the edges incident to it. The resulting degrees are {A: 2, B: 2, C: 3, D: 1}, which form the degree sequence [3, 2, 2, 1]. The total number of vertices identified is 4 and the total number of edges is 4. These values allow for the analysis of the graph's structure, such as identifying regularity or connectivity patterns.

Logic (9)

Contradiction test — `logic_contradiction`

Whether a formula is false under every assignment (unsatisfiable), with a satisfying assignment if not.

$$\forall v:\ f(v) = 0$$

symbol		meaning	SI unit
`formula`	in	the formula (operators ~ & | ^ -> <->, or words)	—
`contradiction`	out	true when no assignment satisfies the formula	—
`true_rows`	out	number of satisfying assignments	—
`witness`	out	the satisfying assignment with the smallest row number (none for a contradiction)	—

Applies when: at most 12 variables.

```
$ discrete-math solve logic_contradiction "formula=p & ~p"
contradiction = true
true_rows = 0
witness = null
  by: Contradiction test  (exhaustive truth table)
  formula = p & ~p
  assumes: at most 12 variables
```

What the worked example means: This calculation determines whether a logical formula is unsatisfiable by checking all possible variable assignments. The result shows that there are 0 satisfying assignments for the given input. If this number is zero, the output confirms it is a contradiction; otherwise, a witness assignment with the smallest row number is provided.

Evaluate a propositional formula — `logic_evaluate`

The truth value of a propositional formula under a given assignment of true/false to its variables.

$$p \to q \equiv \neg p \lor q$$

symbol		meaning	SI unit
`formula`	in	the formula, e.g. (p -> q) & ~r (operators ~ & | ^ -> <->, or not/and/or/xor/implies/iff)	—
`assignment`	in	a truth value for every variable, e.g. 'p = true, q = 0, r = false'	—
`value`	out	the formula's truth value	—

Applies when: every variable gets a value and no other name is given; precedence: ~ binds tightest, then &, ^, |, -> (right-associative), <-> loosest.

```
$ discrete-math solve logic_evaluate "formula=(p -> q) & ~r" "assignment=p = true, q = false, r =
false"
value = false
  by: Evaluate a propositional formula  (recursive-descent parse into a syntax tree, evaluated
bottom-up)
  assignment = p = true, q = false, r = false
  formula = (p -> q) & ~r
  assumes: every variable gets a value and no other name is given; precedence: ~ binds tightest,
then &, ^, |, -> (right-associative), <-> loosest
```

What the worked example means: This calculation determines the logical truth value of a propositional formula based on specific variable assignments. The system parses the expression into a syntax tree and evaluates it from the bottom up, applying standard operator precedence where negation binds tightest and equivalence is loosest. For the input formula (p -> q) & ~r and the given values in p = true, q = false, r = false, the resulting evaluation yields no, indicating whether the entire statement holds true or false under these conditions.

Logical equivalence of two formulas — `logic_equivalence`

Whether two formulas have the same truth value under every assignment, with a distinguishing assignment if not.

$$f \equiv g \iff \models f \leftrightarrow g$$

symbol		meaning	SI unit
`formula1`	in	the first formula	—
`formula2`	in	the second formula	—
`equivalent`	out	true when the formulas agree on every assignment	—
`differing_rows`	out	number of assignments where they differ	—
`counterexample`	out	the first distinguishing assignment (none if equivalent)	—

Applies when: at most 12 variables in total; rows range over the union of both formulas' variables.

```
$ discrete-math solve logic_equivalence "formula1=(p -> q)" "formula2=(~p | q)"
equivalent = true
differing_rows = 0
counterexample = null
  by: Logical equivalence of two formulas  (compare the two truth tables)
  formula1 = (p -> q)
  formula2 = (~p | q)
  assumes: at most 12 variables in total; rows range over the union of both formulas' variables
```

What the worked example means: The formulas are equivalent if they agree on every assignment, with yes indicating the result and 0 rows showing disagreement.

Minimal product of sums (CNF) — `minimal_cnf`

A minimum conjunctive normal form of a formula: fewest clauses, then fewest literals.

$$f = \prod_{\text{cover of } \bar{f}} \sum \bar{\ell}_i$$

symbol		meaning	SI unit
`formula`	in	the formula (at most 6 variables)	—
`expression`	out	a minimal product of sums (clauses joined by &)	—
`clauses`	out	number of clauses	—
`literals`	out	number of literals	—

Applies when: at most 6 variables; cost: fewest clauses first, then fewest literals.

```
$ discrete-math solve minimal_cnf "formula=(A | B) & (A | ~B | C)"
expression = (A | B) & (A | C)
clauses = 2
literals = 4
  by: Minimal product of sums (CNF)  (minimal sum of products of the negation, complemented (De
Morgan))
  formula = (A | B) & (A | ~B | C)
  assumes: at most 6 variables; cost: fewest clauses first, then fewest literals
```

In the worked example, formula = (A | B) & (A | ~B | C) go through the equation above and give expression = (A | B) & (A | C), clauses = 2, literals = 4. It applies only when at most 6 variables; cost: fewest clauses first, then fewest literals: outside that it says what is wrong instead of returning a number.

Minimal sum of products (DNF) — `minimal_dnf`

A minimum disjunctive normal form of a formula: fewest product terms, then fewest literals.

$$f = \sum_{\text{cover}} \prod \ell_i$$

symbol		meaning	SI unit
`formula`	in	the formula (at most 6 variables)	—
`expression`	out	a minimal sum of products (terms joined by |, literals by &)	—
`terms`	out	number of product terms	—
`literals`	out	number of literals	—

Applies when: at most 6 variables; cost: fewest terms first, then fewest literals; when several minimal forms exist one is shown (all have these counts).

```
$ discrete-math solve minimal_dnf "formula=(A & B) | (A & ~B & C)"
expression = A & B | A & C
terms = 2
literals = 4
  by: Minimal sum of products (DNF)  (Quine-McCluskey prime implicants, then an exact branch-and-
bound minimum cover)
  formula = (A & B) | (A & ~B & C)
  assumes: at most 6 variables; cost: fewest terms first, then fewest literals; when several
minimal forms exist one is shown (all have these counts)
```

What the worked example means: This calculation determines the simplest disjunctive normal form for the provided logical formula. By applying the Quine-McCluskey algorithm to identify prime implicants followed by a branch-and-bound search, it finds an expression with the fewest possible product terms. If multiple forms share this minimum term count, it selects one with the fewest literals. The resulting simplified logic is A & B | A & C, which uses 2 distinct product terms and contains 4 individual literals in total.

Minimal sum of products from minterms (with don't-cares) — `minimal_sop_minterms`

The minimal sum of products of a Boolean function given by its minterms and optional don't-care rows (Karnaugh-map problems).

$$F(A,B,C) = \sum m(\ldots) + d(\ldots)$$

symbol		meaning	SI unit
`minterms`	in	rows where the function is 1	—
`variables`	in	number of variables (1..6), named A, B, C, ...	—
`dont_cares`	in	rows whose value does not matter (optional) (optional)	—

symbol		meaning	SI unit
`expression`	out	a minimal sum of products over A, B, C, ... (A is the most significant bit)	—
`terms`	out	number of product terms	—
`literals`	out	number of literals	—

Applies when: 1..6 variables named A, B, C, D, E, F with A the most significant bit of the row number; cost: fewest terms first, then fewest literals; don't-cares may be used or not.

```
$ discrete-math solve minimal_sop_minterms "minterms=[1, 3, 5, 7]" variables=3
expression = C
terms = 1
literals = 1
  by: Minimal sum of products from minterms (with don't-cares)  (Quine-McCluskey prime implicants
(don't-cares included), exact minimum cover of the minterms)
  assumes: 1..6 variables named A, B, C, D, E, F with A the most significant bit of the row number;
cost: fewest terms first, then fewest literals; don't-cares may be used or not
```

What the worked example means: This calculation determines the simplest Boolean expression that covers all specified true rows while optionally utilizing don't-care conditions. By applying the Quine-McCluskey algorithm, it identifies prime implicants and selects an exact minimum cover based on a cost function that prioritizes fewer product terms first, then fewer total literals. The resulting expression C represents this optimal form, using variables where A is the most significant bit, with a final count of 1 terms and 1 literals.

Satisfiability of a propositional formula — `logic_satisfiable`

Whether some assignment makes the formula true, how many do, and the first such assignment.

$$\exists v : \ f(v) = 1$$

symbol		meaning	SI unit
`formula`	in	the formula (operators ~ & | ^ -> <->, or words)	—
`satisfiable`	out	true when some assignment satisfies the formula	—
`models`	out	number of satisfying assignments	—
`first_model`	out	the satisfying assignment with the smallest row number (none if unsatisfiable)	—

Applies when: at most 12 variables; rows are numbered with the first (alphabetical) variable as the most significant bit.

```
$ discrete-math solve logic_satisfiable "formula=(p | q) & ~p & ~q"
satisfiable = false
models = 0
first_model = null
  by: Satisfiability of a propositional formula  (exhaustive truth table)
  formula = (p | q) & ~p & ~q
  assumes: at most 12 variables; rows are numbered with the first (alphabetical) variable as the
most significant bit
```

In the worked example, formula = (p | q) & ~p & ~q go through the equation above and give satisfiable = False, models = 0, first_model = None. It applies only when at most 12 variables; rows are numbered with the

first (alphabetical) variable as the most significant bit: outside that it says what is wrong instead of returning a number.

Tautology test — `logic_tautology`

Whether a formula is true under every assignment, and the first assignment that falsifies it if not.

$$\models f \iff \forall v : \; f(v) = 1$$

symbol		meaning	SI unit
`formula`	in	the formula (operators ~ & | ^ -> <->, or words)	—
`tautology`	out	true when every assignment satisfies the formula	—
`false_rows`	out	number of falsifying assignments	—
`counterexample`	out	the falsifying assignment with the smallest row number (none for a tautology)	—

Applies when: at most 12 variables.

```
$ discrete-math solve logic_tautology "formula=(p -> q) | (q -> p)"
tautology = true
false_rows = 0
counterexample = null
  by: Tautology test  (exhaustive truth table)
  formula = (p -> q) | (q -> p)
  assumes: at most 12 variables
```

What the worked example means: The formula (p -> q) | (q -> p) is a tautology if yes is true, meaning it holds for all assignments.

Truth table and minterms — `truth_table`

The full truth table of a propositional formula, the number of true rows and its minterms.

$$f = \sum m(\text{true rows})$$

symbol		meaning	SI unit
`formula`	in	the formula (operators ~ & | ^ -> <->, or words)	—
`variables`	out	the variables in alphabetical order	—
`values`	out	the formula's value on rows 0, 1, ... as a 0/1 string	—
`true_rows`	out	number of true rows	—
`minterms`	out	indices of the true rows	—

Applies when: at most 12 variables; row r assigns the binary digits of r to the variables, the first (alphabetically) variable being the most significant bit.

```
$ discrete-math solve truth_table "formula=(p -> q) & (q -> r)"
variables = ["p","q","r"]
values = 11010001
true_rows = 4
minterms = [0,1,3,7]
  by: Truth table and minterms  (evaluate the syntax tree on every assignment)
  formula = (p -> q) & (q -> r)
  assumes: at most 12 variables; row r assigns the binary digits of r to the variables, the first
(alphabetically) variable being the most significant bit
```

What the worked example means: This calculation evaluates the propositional formula (p -> q) & (q -> r) by constructing its full truth table. The variables are identified as [p, q, r] and sorted alphabetically to determine bit significance, with the first variable acting as the most significant bit. Each row index corresponds to a unique binary assignment of these variables. The resulting sequence of outcomes is captured in 11010001, where each character represents the formula's truth value for that specific input combination. This data gives the total count of true rows, which is 4, and the specific row numbers where the formula holds true, known as the minterms, shown here as [0, 1, 3, 7].

Modular arithmetic (8)

Chinese remainder theorem — `crt`

The x with x ≡ r_i (mod m_i) for every i, unique modulo lcm(m_i); moduli need not be coprime.

$$x \equiv r_i \pmod{m_i},\ i = 1..k \ \Rightarrow\ x \bmod \operatorname{lcm}(m_1, \ldots, m_k)$$

symbol		meaning	SI unit
`residues`	in	the residues r_i	—
`moduli`	in	the moduli m_i (each at least 1)	—
`x`	out	the smallest non-negative solution	—
`modulus`	out	lcm of the moduli: the solution is unique modulo it	—

Applies when: one residue per modulus; the system is consistent (r_i ≡ r_j modulo gcd(m_i, m_j)).

```
$ discrete-math solve crt "residues=[2, 3, 2]" "moduli=[3, 5, 7]"
x = 23
modulus = 105
  by: Chinese remainder theorem  (merge the congruences pairwise with the extended Euclidean
algorithm (works for non-coprime moduli))
  assumes: one residue per modulus; the system is consistent (r_i ≡ r_j modulo gcd(m_i, m_j))
```

What the worked example means: This calculation determines the unique integer x that satisfies all given congruences, expressed as 23. The result is valid modulo 105, which represents the least common multiple of the input moduli. The method merges the congruence pairs using the extended Euclidean algorithm, allowing for non-coprime moduli provided the system remains consistent.

Discrete logarithm — `discrete_log`

The smallest x >= 0 with base^x ≡ target (mod m).

$$g^x \equiv h \pmod{m}, \quad x = i\lceil\sqrt{m}\rceil + j$$

symbol		meaning	SI unit
`base`	in	the base g	—
`target`	in	the value h	—
`modulus`	in	the modulus m (2..10^12)	—
`x`	out	the smallest non-negative exponent with g^x ≡ h (mod m)	—

Applies when: gcd(base, m) = 1; a solution exists.

Run `discrete-math info discrete_log` for its inputs, then `discrete-math solve discrete_log name=value ...`.

What the worked example means: This calculation determines the discrete logarithm, which is the smallest non-negative integer x such that raising the base to the power of x yields the target value modulo the given modulus. The baby-step giant-step algorithm searches efficiently for this exponent by precomputing small powers and checking against larger steps. The result 4 represents the specific power needed to transform the base 3 into the target 13 under modular arithmetic defined by 17. This process assumes that the base and modulus are coprime and that a valid solution exists within the specified range.

Linear congruence ax ≡ b (mod m) — `linear_congruence`

All solutions of a*x ≡ b (mod m): how many there are modulo m, the smallest, and their spacing.

$$ax \equiv b \pmod{m} \iff x \equiv \tfrac{b}{g}\left(\tfrac{a}{g}\right)^{-1} \pmod{\tfrac{m}{g}},\ g = \gcd(a,m) \mid b$$

symbol		meaning	SI unit
`a`	in	coefficient of x	—
`b`	in	right-hand side	—
`m`	in	modulus (at least 1)	—
`smallest`	out	smallest non-negative solution (none when there is no solution)	—
`count`	out	number of solutions modulo m (gcd(a, m) when it divides b, else 0)	—
`step`	out	m / gcd(a, m): solutions are smallest + k*step	—

Applies when: m >= 1; solvable exactly when gcd(a, m) divides b.

```
$ discrete-math solve linear_congruence a=3 b=4 m=7
smallest = 6
count = 1
step = 7
  by: Linear congruence ax ≡ b (mod m)  (divide by g = gcd(a, m), then multiply by the inverse of
a/g modulo m/g)
  assumes: m >= 1; solvable exactly when gcd(a, m) divides b
```

What the worked example means: This calculation determines the integer values of x that satisfy the linear congruence equation. By computing the greatest common divisor of the coefficient and the modulus, the algorithm checks for solvability. If a solution exists, it finds the smallest non-negative value, which is reported

as 6. The total number of distinct solutions within one full cycle of the modulus is given by 1. These solutions are evenly spaced, with the distance between consecutive answers defined as 7. This process transforms the original problem into a simpler modular inverse calculation, ensuring all valid integers are identified without listing every single possibility.

Modular exponentiation — `mod_pow`

a^e mod m by repeated squaring (also the last k digits of a power).

$$a^e \bmod m, \quad a^{2k} = (a^k)^2,\ a^{2k+1} = a\,(a^k)^2$$

symbol		meaning	SI unit
`base`	in	the base a	—
`exponent`	in	the exponent e (at least 0)	—
`modulus`	in	the modulus m (at least 1)	—
`result`	out	a^e mod m, between 0 and m - 1	—

Applies when: e >= 0; m >= 1.

```
$ discrete-math solve mod_pow base=3 exponent=200 modulus=13
result = 9
  by: Modular exponentiation  (binary (square-and-multiply) exponentiation, reducing modulo m at
every step)
  assumes: e >= 0; m >= 1
```

What the worked example means: This calculation determines the remainder when the base 3 is raised to the exponent 200 and then divided by the modulus 13. The method employs binary square-and-multiply exponentiation, reducing intermediate values modulo 13 at each step to maintain efficiency and numerical stability. The final computed value is 9, which represents the unique integer between zero and one less than the modulus that satisfies the congruence relation.

Modular multiplicative inverse — `mod_inverse`

The x in 0..m-1 with a*x ≡ 1 (mod m); it exists exactly when gcd(a, m) = 1.

$$ax \equiv 1 \pmod{m}$$

symbol		meaning	SI unit
`a`	in	the integer to invert	—
`m`	in	the modulus (at least 2)	—
`inverse`	out	a^{-1} mod m	—

Applies when: m >= 2; gcd(a, m) = 1 (otherwise no inverse exists).

```
$ discrete-math solve mod_inverse a=3 m=11
inverse = 4
  by: Modular multiplicative inverse  (extended Euclidean algorithm)
  assumes: m >= 2; gcd(a, m) = 1 (otherwise no inverse exists)
```

What the worked example means: This calculation determines the integer 4 such that when multiplied by 3, the product leaves a remainder of one when divided by 11. The result exists only if 3 and 11 share no common factors other than one. If this condition holds, 4 serves as the unique solution in the range from zero to one less than 11, effectively acting as the reciprocal for modular arithmetic operations involving these specific values.

Multiplicative order — `multiplicative_order`

The smallest k >= 1 with a^k ≡ 1 (mod n).

$$\mathrm{ord}_n(a) = \min\{k \geq 1 : a^k \equiv 1 \pmod{n}\},\ \mathrm{ord}_n(a) \mid \lambda(n)$$

symbol		meaning	SI unit
`a`	in	the integer a (coprime to n)	—
`n`	in	the modulus (at least 2)	—
`order`	out	ord_n(a)	—

Applies when: gcd(a, n) = 1.

```
$ discrete-math solve multiplicative_order a=2 n=7
order = 3
  by: Multiplicative order  (start from the Carmichael exponent lambda(n) and divide out prime
factors while a^(k/p) ≡ 1)
  assumes: gcd(a, n) = 1
```

What the worked example means: This calculation determines the smallest positive integer exponent, denoted as 3, such that raising the base 2 to this power yields a result congruent to one modulo 7. The process begins by establishing the Carmichael exponent for the modulus and then systematically reducing it by dividing out prime factors until the condition is no longer met. This value represents the length of the cycle generated by the base within the modular arithmetic group defined by the modulus.

Remainder modulo m (Euclidean division) — `mod_reduce`

a mod m as the least non-negative residue, with the floor quotient: a = q*m + r, 0 <= r < m.

$$a = qm + r, \quad 0 \leq r < m$$

symbol		meaning	SI unit
`a`	in	the integer to reduce	—
`m`	in	the modulus (at least 1)	—
`remainder`	out	r with 0 <= r < m	—
`quotient`	out	q = floor(a/m)	—

Applies when: m >= 1; the remainder is never negative, also for negative a.

```
$ discrete-math solve mod_reduce a=17 m=5
remainder = 2
quotient = 3
  by: Remainder modulo m (Euclidean division)  (floor division)
  assumes: m >= 1; the remainder is never negative, also for negative a
```

What the worked example means: This calculation finds the Euclidean remainder when the integer 17 is divided by the modulus 5. The result ensures that the remainder 2 is always non-negative and strictly less than 5, while the quotient 3 represents the floor division of the two values. This method provides a consistent modular arithmetic result, even when dealing with negative integers.

Smallest primitive root modulo a prime — `primitive_root`

The smallest g whose powers run through every non-zero residue modulo the prime p.

$$g^{(p-1)/q} \not\equiv 1 \pmod{p} \;\; \forall q \mid p-1$$

symbol		meaning	SI unit
`p`	in	a prime	—
`root`	out	the smallest primitive root modulo p	—

Applies when: p prime (primitive roots also exist for 4, p^k and 2p^k, which this capability does not take).

```
$ discrete-math solve primitive_root p=23
root = 5
  by: Smallest primitive root modulo a prime  (g is a primitive root iff g^((p-1)/q) ≡/1 for every
prime q dividing p - 1; test g = 2, 3, ...)
  assumes: p prime (primitive roots also exist for 4, p^k and 2p^k, which this capability does not
take)
```

What the worked example means: This calculation identifies the smallest integer whose powers generate all non-zero residues modulo the prime number 23. The result, 5, serves as a generator for the multiplicative group of integers modulo 23. This means that by repeatedly multiplying 5 by itself and taking the remainder after division by 23, one can produce every number from one up to one less than 23 exactly once before repeating. Finding such a root is fundamental in number theory and cryptography.

Number representations (2)

Base (radix) conversion — `base_convert`

Rewrite an integer from one base (2..36) to another, with its decimal value.

$$(d_k \ldots d_0)_b = \sum_i d_i \, b^i$$

symbol		meaning	SI unit
`digits`	in	the number written in the source base (digits 0-9 then A-Z)	—

symbol		meaning	SI unit
`from_base`	in	the source base (2..36)	—
`to_base`	in	the target base (2..36)	—
`result`	out	the number written in the target base (upper-case digits)	—
`decimal`	out	its value in base 10	—

Applies when: 2 <= bases <= 36; every digit is valid in the source base; a leading minus sign is kept.

```
$ discrete-math solve base_convert digits=1011 from_base=2 to_base=10
result = 11
decimal = 11
  by: Base (radix) conversion  (Horner evaluation in the source base, then repeated division by the
target base)
  digits = 1011
  assumes: 2 <= bases <= 36; every digit is valid in the source base; a leading minus sign is kept
```

What the worked example means: This calculation rewrites the integer represented by the input digits from its source base into the target base. The process first evaluates the polynomial sum of each digit multiplied by powers of the source base to determine the standard decimal value, which is reported as 11. Then, this decimal value is repeatedly divided by the target base, collecting remainders to form the new representation 11. This method ensures that the magnitude of the number remains unchanged while only its symbolic notation shifts between radix systems.

Two's complement representation — `twos_complement`

The b-bit two's complement bit pattern of a signed integer, and that pattern read as an unsigned number.

$$\text{pattern}(n) = n \bmod 2^b$$

symbol		meaning	SI unit
`n`	in	the signed integer	—
`bits`	in	the word length b (2..64)	—
`bits_text`	out	the b-bit pattern	—
`unsigned`	out	the pattern read as an unsigned integer (n mod 2^b)	—

Applies when: -2^(b-1) <= n < 2^(b-1).

```
$ discrete-math solve twos_complement n=-5 bits=8
bits_text = 11111011
unsigned = 251
  by: Two's complement representation  (non-negative: binary padded to b bits; negative: invert the
bits of |n| and add 1)
  assumes: -2^(b-1) <= n < 2^(b-1)
```

What the worked example means: This calculation determines the fixed-width binary representation of a signed integer. Given the value -5 and a word length of 8, the system produces the bit sequence 11111011. This same pattern, when interpreted as an unsigned number, equals 251. The method ensures that positive

numbers are padded with zeros while negative numbers are represented by inverting the bits of their absolute value and adding one, maintaining consistent arithmetic properties across the entire range.

Number theory (21)

Carmichael function — `carmichael_lambda`

lambda(n): the smallest m with a^m ≡ 1 (mod n) for every a coprime to n (the exponent of the unit group).

$$\lambda(n) = \operatorname{lcm}\left(\lambda(p_1^{e_1}), \dots\right),\ \lambda(p^e) = p^{e-1}(p-1)$$

symbol		meaning	SI unit
`n`	in	a positive integer	—
`lambda`	out	Carmichael's lambda(n)	—

Applies when: n >= 1.

```
$ discrete-math solve carmichael_lambda n=15
lambda = 4
  by: Carmichael function  (lcm over prime powers: lambda(2^e) = 2^(e-2) for e >= 3, else phi(p^e))
  assumes: n >= 1
```

What the worked example means: This calculation determines the Carmichael lambda value, which is the smallest positive integer m such that a raised to the power of m is congruent to one modulo n for every integer a coprime to n. For the input 15, the result is 4. This value represents the exponent of the multiplicative group of integers modulo n and often provides a tighter bound than Euler's totient function in cryptographic contexts.

Continued fraction of a rational number — `cf_rational`

The finite simple continued fraction [a0; a1, ..., an] of p/q (canonical form: last term >= 2).

$$\frac{p}{q} = a_0 + \cfrac{1}{a_1 + \cfrac{1}{\ddots + \cfrac{1}{a_n}}}$$

symbol		meaning	SI unit
`numerator`	in	the numerator p	—
`denominator`	in	the denominator q (at least 1)	—
`terms`	out	the partial quotients [a0, a1, ..., an]	—
`length`	out	number of partial quotients	—

Applies when: q >= 1; a0 = floor(p/q) (negative for negative p); canonical: the last term is at least 2.

```
$ discrete-math solve cf_rational numerator=415 denominator=93
terms = [4,2,6,7]
length = 4
  by: Continued fraction of a rational number  (Euclidean algorithm: a_k = floor(p/q), then (p, q)
<- (q, p - a_k q))
  assumes: q >= 1; a0 = floor(p/q) (negative for negative p); canonical: the last term is at least
2
```

What the worked example means: This calculation determines the finite simple continued fraction expansion for the rational number formed by 415 divided by 93. By applying the Euclidean algorithm, the process extracts a sequence of integer partial quotients, resulting in the canonical form [4, 2, 6, 7]. The length of this sequence is recorded as 4, indicating how many steps were required to reduce the fraction to its simplest continued fraction representation.

Continued fraction of a square root — `cf_sqrt`

The periodic continued fraction sqrt(d) = [a0; (a1, ..., ak)] of a non-square d, and its period length.

$$\sqrt{d} = [a_0; \overline{a_1, \ldots, a_{k-1}, 2a_0}]$$

symbol		meaning	SI unit
`d`	in	a positive non-square integer	—
`a0`	out	integer part floor(sqrt(d))	—
`period`	out	the repeating block (ends with 2*a0)	—
`period_length`	out	length of the period	—

Applies when: d not a perfect square.

```
$ discrete-math solve cf_sqrt d=23
a0 = 4
period = [1,3,1,8]
period_length = 4
  by: Continued fraction of a square root  (the exact recurrence m' = den*a - m, den' = (d -
m'^2)/den, a' = floor((a0 + m')/den') until a' = 2*a0)
  assumes: d not a perfect square
```

What the worked example means: This calculation determines the periodic continued fraction expansion for the square root of the non-square integer 23. The result identifies the initial term as 4, which is the floor of the square root, and reveals that the subsequent terms repeat in a cycle known as the period. This period ends with twice the initial term, specifically 4 multiplied by two, and has a total length of 4 terms. Such expansions are fundamental in number theory, particularly for solving quadratic Diophantine equations.

Divisors, divisor count and divisor sum — `divisor_functions`

All positive divisors of n, their number tau(n), their sum sigma(n), and whether n is perfect, abundant or deficient.

$$\tau(n) = \prod (e_i + 1), \qquad \sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}$$

symbol		meaning	SI unit
`n`	in	a positive integer	—
`count`	out	number of divisors tau(n)	—
`sum`	out	sum of divisors sigma(n)	—
`proper_sum`	out	sum of the proper divisors sigma(n) - n	—
`classification`	out	perfect, abundant or deficient	—
`divisors`	out	every divisor, ascending	—

Applies when: n a whole number, 1 <= n <= 10^12.

```
$ discrete-math solve divisor_functions n=360
count = 24
sum = 1170
proper_sum = 810
classification = abundant
divisors = [1,2,3,4,5,6,8,9,10,12,15,18,20,24,30,36,40,45,60,72,90,120,180,360]
  by: Divisors, divisor count and divisor sum  (divisors generated from the prime factorization;
tau = prod(e + 1), sigma = sum of the divisors)
  assumes: n a whole number, 1 <= n <= 10^12
```

What the worked example means: The algorithm identifies all positive divisors of the integer 360, resulting in a total count of 24 and a sum of 1170. By comparing the sum of proper divisors, which equals 810, against the original number, the tool classifies 360 as abundant. This process relies on generating divisors from the prime factorization to ensure accuracy for integers within the specified range.

Euler's totient function — `euler_phi`

phi(n): how many integers in 1..n are coprime to n.

$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right)$$

symbol		meaning	SI unit
`n`	in	a positive integer	—
`phi`	out	Euler's totient phi(n)	—

Applies when: n a whole number, 1 <= n <= 10^15.

```
$ discrete-math solve euler_phi n=36
phi = 12
  by: Euler's totient function  (product formula over the distinct prime factors of n)
  assumes: n a whole number, 1 <= n <= 10^15
```

What the worked example means: This calculation determines the number of positive integers less than or equal to 36 that are coprime to it, denoted as 12. The result is derived by applying the product formula over the distinct prime factors of the input. A value of 12 indicates exactly how many such relatively prime numbers exist within the range from one to 36, which is a fundamental concept in modular arithmetic and cryptography.

Extended Euclidean algorithm (Bezout coefficients) — `extended_gcd`

gcd(a, b) and integers x, y with a*x + b*y = gcd(a, b).

$$ax + by = \gcd(a, b)$$

symbol		meaning	SI unit
`a`	in	first positive integer	—
`b`	in	second positive integer	—
`gcd`	out	gcd(a, b)	—
`x`	out	coefficient of a from the extended Euclidean algorithm	—
`y`	out	coefficient of b from the extended Euclidean algorithm	—
`x_min`	out	the coefficient of a with 0 <= x_min < b/gcd	—
`y_min`	out	the matching coefficient of b	—

Applies when: a, b whole numbers >= 1; (x, y) are the coefficients the standard extended Euclidean algorithm produces; (x_min, y_min) is the solution with the smallest non-negative x.

```
$ discrete-math solve extended_gcd a=240 b=46
gcd = 2
x = -9
y = 47
x_min = 14
y_min = -73
  by: Extended Euclidean algorithm (Bezout coefficients)  (extended Euclidean algorithm (back-substitution kept as running coefficients))
  assumes: a, b whole numbers >= 1; (x, y) are the coefficients the standard extended Euclidean algorithm produces; (x_min, y_min) is the solution with the smallest non-negative x
```

What the worked example means: This calculation determines the greatest common divisor 2 of the two positive integers 240 and 46, along with integer coefficients -9 and 47 that satisfy the linear combination equation. The coefficient -9 multiplies the first input, while 47 multiplies the second, ensuring their weighted sum equals the greatest common divisor. Additionally, the solution set includes a normalized pair where the coefficient for the first input is constrained to be non-negative and less than the quotient of the second input divided by the greatest common divisor.

Greatest common divisor and least common multiple — `gcd_lcm`

The gcd and lcm of two or more positive integers (Euclid's algorithm).

$$\gcd(a, b) = \gcd(b,\ a \bmod b), \qquad \operatorname{lcm}(a, b) = \frac{ab}{\gcd(a, b)}$$

symbol		meaning	SI unit
`numbers`	in	the positive integers (two or more)	—
`gcd`	out	greatest common divisor	—
`lcm`	out	least common multiple	—

Applies when: two or more whole numbers, each at least 1.

```
$ discrete-math solve gcd_lcm "numbers=[84, 36]"
gcd = 12
lcm = 252
  by: Greatest common divisor and least common multiple  (Euclid's algorithm gcd(a, b) = gcd(b, a
mod b), folded over the list; lcm(a, b) = a / gcd(a, b) * b)
  assumes: two or more whole numbers, each at least 1
```

What the worked example means: This calculation determines the largest number that divides evenly into all provided inputs, known as the greatest common divisor, and the smallest number that is a multiple of all provided inputs, known as the least common multiple. The result for the greatest common divisor is 12, while the result for the least common multiple is 252. These values are derived using Euclid's algorithm, which iteratively finds remainders until reaching zero, followed by a formula linking the two metrics.

Integer square root and perfect squares — `integer_sqrt`

floor(sqrt(n)) computed exactly, the remainder n - floor(sqrt(n))^2, and whether n is a perfect square.

$$r = \lfloor \sqrt{n} \rfloor \iff r^2 \le n < (r+1)^2$$

symbol		meaning	SI unit
`n`	in	a whole number (at least 0)	—
`isqrt`	out	floor(sqrt(n))	—
`remainder`	out	n - isqrt^2	—
`is_square`	out	true when n is a perfect square	—

Applies when: n >= 0.

```
$ discrete-math solve integer_sqrt n=1000
isqrt = 31
remainder = 39
is_square = false
  by: Integer square root and perfect squares  (exact integer Newton iteration (no floating point))
  assumes: n >= 0
```

What the worked example means: This calculation determines the largest whole number whose square does not exceed the input value. For the given number 1000, the result is 31, which satisfies the condition that its square is less than or equal to the input while the next integer's square exceeds it. The difference between the input and the square of this result is reported as 39. If the remainder is zero, the input is identified as a perfect square.

Jacobi symbol — `jacobi_symbol`

(a/n) for an odd positive n, the product of Legendre symbols over the prime factors of n.

$$\left(\frac{a}{n}\right) = \prod_i \left(\frac{a}{p_i}\right)^{e_i}, \quad n = \prod p_i^{e_i}$$

symbol		meaning	SI unit
`a`	in	the integer a	—
`n`	in	an odd positive integer	—
`symbol`	out	the Jacobi symbol (a/n): -1, 0 or 1	—

Applies when: n odd and positive.

```
$ discrete-math solve jacobi_symbol a=1001 n=9907
symbol = -1
  by: Jacobi symbol  (quadratic reciprocity with the supplementary law for 2 (no factoring needed))
  assumes: n odd and positive
```

What the worked example means: This calculation determines the Jacobi symbol for the integer 1001 with respect to the odd positive modulus 9907. The result, -1, indicates whether 1001 is a quadratic residue relative to the prime factors of 9907. Unlike the Legendre symbol, a value of one here does not guarantee that a square root exists modulo 9907, but it provides essential information about the algebraic structure of the congruence.

Legendre symbol — `legendre_symbol`

(a/p) for an odd prime p: 1 if a is a non-zero quadratic residue mod p, -1 if not, 0 if p divides a.

$$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}$$

symbol		meaning	SI unit
`a`	in	the integer a	—
`p`	in	an odd prime	—
`symbol`	out	the Legendre symbol (a/p): -1, 0 or 1	—

Applies when: p an odd prime.

```
$ discrete-math solve legendre_symbol a=5 p=11
symbol = 1
  by: Legendre symbol  (quadratic reciprocity with the supplementary law for 2 (no factoring
needed))
  assumes: p an odd prime
```

What the worked example means: This calculation determines the Legendre symbol, which indicates whether an integer is a quadratic residue modulo an odd prime. Given the input integer 5 and the odd prime modulus 11, the resulting value 1 reveals the relationship between them. A result of one means that 5 has a square root modulo 11, while negative one means it does not. If the result is zero, then 5 is divisible by 11.

This assessment relies on properties of modular arithmetic and quadratic reciprocity to classify the solvability of congruences without explicitly finding roots.

Linear Diophantine equation ax + by = c — `linear_diophantine`

Whether a*x + b*y = c has integer solutions, and the general solution x = x0 + (b/g) t, y = y0 - (a/g) t.

$$ax + by = c \text{ solvable} \iff \gcd(a, b) \mid c; \quad x = x_0 + \tfrac{b}{g}t,\ y = y_0 - \tfrac{a}{g}t$$

symbol		meaning	SI unit
`a`	in	coefficient of x (non-zero)	—
`b`	in	coefficient of y (non-zero)	—
`c`	in	right-hand side	—
`solvable`	out	true when gcd(a, b) divides c	—
`x0`	out	the solution with the smallest non-negative x	—
`y0`	out	the matching y	—
`x_step`	out	b/g: the step of x between solutions	—
`y_step`	out	-a/g: the step of y between solutions	—

Applies when: a, b non-zero integers; solutions exist exactly when gcd(a, b) divides c.

```
$ discrete-math solve linear_diophantine a=12 b=18 c=30
solvable = true
x0 = 1
y0 = 1
x_step = 3
y_step = -2
  by: Linear Diophantine equation ax + by = c  (extended Euclidean algorithm, scaled by c/g,
reduced to the smallest non-negative x)
  assumes: a, b non-zero integers; solutions exist exactly when gcd(a, b) divides c
```

What the worked example means: This calculation determines if integer solutions exist for the linear Diophantine equation defined by coefficients 12 and 18 and constant term 30. It checks if the greatest common divisor of the coefficients divides the constant. If solvable, it uses the extended Euclidean algorithm to find a particular solution, then adjusts it to ensure x is the smallest non-negative value possible. The result shows that x equals 1 and y equals 1, with subsequent solutions stepping by 3 in x and -2 in y.

Möbius function — `mobius`

mu(n): 0 if a square divides n, otherwise (-1)^(number of prime factors).

$$\mu(n) = \begin{cases} (-1)^k & n = p_1 \cdots p_k \text{ squarefree} \\ 0 & p^2 \mid n \end{cases}$$

symbol		meaning	SI unit
`n`	in	a positive integer	—
`mu`	out	the Möbius function mu(n)	—

symbol		meaning	SI unit
`squarefree`	out	true when no square > 1 divides n	—

Applies when: n >= 1.

```
$ discrete-math solve mobius n=30
mu = -1
squarefree = true
  by: Möbius function  (prime factorization)
  assumes: n >= 1
```

What the worked example means: This calculation determines the Möbius function for the positive integer 30. The result, -1, reflects the parity of distinct prime factors or indicates the presence of a squared factor. The boolean flag yes confirms whether any perfect square greater than one divides the input.

Next prime — `next_prime`

The smallest prime strictly greater than n.

$$\min\{p > n : p \text{ prime}\}$$

symbol		meaning	SI unit
`n`	in	the whole number n (at least 0)	—
`prime`	out	the smallest prime p > n	—

Applies when: n >= 0; strictly greater: the next prime after a prime p is not p.

```
$ discrete-math solve next_prime n=100
prime = 101
  by: Next prime  (test n + 1, n + 2, ... with deterministic Miller-Rabin)
  assumes: n >= 0; strictly greater: the next prime after a prime p is not p
```

What the worked example means: This calculation determines the next available prime number by testing integers sequentially starting from one above 100. The result, 101, represents the minimal integer that satisfies the primality condition while being strictly larger than the input value. This process ensures that even if 100 itself is a prime number, the output will be the subsequent distinct prime.

Pell's equation (fundamental solution) — `pell_equation`

The smallest positive solution (x, y) of x^2 - d y^2 = 1 for a non-square d.

$$x^2 - d\,y^2 = 1, \quad \frac{x}{y} = [a_0; a_1, \ldots, a_{k-1}]$$

symbol		meaning	SI unit
`d`	in	a non-square positive integer (2..1000)	—
`x`	out	fundamental solution x	—
`y`	out	fundamental solution y	—

Applies when: d not a perfect square; 2 <= d <= 1000.

```
$ discrete-math solve pell_equation d=61
x = 1766319049
y = 226153980
  by: Pell's equation (fundamental solution)  (convergent of the continued fraction of sqrt(d) at
the end of the first period (squared when the period is odd))
  assumes: d not a perfect square; 2 <= d <= 1000
```

What the worked example means: This calculation determines the smallest positive integer pair satisfying the equation x squared minus d times y squared equals one. For the input value 61, the algorithm identifies the convergent of the continued fraction expansion that yields the fundamental solution. The resulting values are x = 1766319049 and y = 226153980, which form the basis for generating all other solutions to this specific instance of the equation.

Previous prime — `previous_prime`

The largest prime strictly less than n.

$$\max\{p < n : p \text{ prime}\}$$

symbol		meaning	SI unit
`n`	in	the whole number n (at least 3)	—
`prime`	out	the largest prime p < n	—

Applies when: n >= 3; strictly less: the prime before a prime p is not p.

```
$ discrete-math solve previous_prime n=100
prime = 97
  by: Previous prime  (test n - 1, n - 2, ... with deterministic Miller-Rabin)
  assumes: n >= 3; strictly less: the prime before a prime p is not p
```

What the worked example means: This calculation identifies the greatest prime number that is smaller than the input integer. By testing integers in descending order starting from one less than the input, the algorithm finds the first value that satisfies the primality test. The result, 97, represents this specific predecessor to 100, providing insight into the local distribution of prime numbers near that value.

Primality test — `is_prime`

Whether n is prime (deterministic Miller-Rabin, exact for every n below 2^64), with its smallest prime factor.

$$a^d \equiv 1 \text{ or } a^{2^r d} \equiv -1 \pmod{n} \text{ for every base } a$$

symbol		meaning	SI unit
`n`	in	the whole number to test	—
`is_prime`	out	true when n is prime	—
`smallest_prime_factor`	out	smallest prime dividing n (n itself when prime; none for n < 2)	—

Applies when: 0 <= n < 2^64 (questions accept n up to 10^15).

```
$ discrete-math solve is_prime n=97
is_prime = true
smallest_prime_factor = 97
  by: Primality test  (trial division by primes up to 37, then Miller-Rabin to the bases 2, 325,
9375, 28178, 450775, 9780504, 1795265022 (deterministic below 2^64); the smallest factor by
Pollard's rho)
  assumes: 0 <= n < 2^64 (questions accept n up to 10^15)
```

What the worked example means: This process evaluates the integer 97 to determine its primality status. It first checks for small divisors and then applies a deterministic Miller-Rabin test using specific bases that guarantee accuracy for numbers below two to the sixty-fourth power. The result indicates whether 97 is prime via yes, and if it is not, the algorithm identifies 97 as the smallest prime number dividing 97. If 97 is itself prime, the smallest factor is reported as 97.

Prime factorization — `prime_factorization`

The prime factorization of n, its largest and smallest prime factors and the number of prime factors.

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$$

symbol		meaning	SI unit
`n`	in	the whole number to factor (at least 2)	—
`factorization`	out	list of [prime, exponent] pairs, primes ascending	—
`distinct_primes`	out	number of distinct prime factors	—
`prime_factors_with_multiplicity`	out	number of prime factors counted with multiplicity	—
`largest_prime_factor`	out	largest prime factor	—
`smallest_prime_factor`	out	smallest prime factor	—
`text`	out	the factorization written as p^e × ...	—

Applies when: n a whole number, 2 <= n <= 10^15.

```
$ discrete-math solve prime_factorization n=360
factorization = [[2,3],[3,2],[5,1]]
distinct_primes = 3
prime_factors_with_multiplicity = 6
largest_prime_factor = 5
smallest_prime_factor = 2
text = 2^3 × 3^2 × 5
  by: Prime factorization  (trial division by small primes, then Pollard's rho with deterministic
Miller-Rabin for the cofactors)
  assumes: n a whole number, 2 <= n <= 10^15
```

What the worked example means: This calculation determines the unique representation of the integer 360 as a product of prime numbers raised to specific powers. The result lists these primes in ascending order along with their exponents, showing that there are 3 distinct prime bases involved. It also identifies the smallest factor 2 and the largest factor 5, providing a complete structural breakdown of the original value.

Prime-counting function — `prime_counting`

pi(x): how many primes are at most x.

$$\pi(x) = \#\{p \le x : p \text{ prime}\}$$

symbol		meaning	SI unit
`x`	in	the bound (0..5,000,000)	—
`count`	out	number of primes p <= x	—

Applies when: 0 <= x <= 5,000,000; 'less than N' counts primes up to N - 1.

```
$ discrete-math solve prime_counting x=99
count = 25
  by: Prime-counting function  (sieve of Eratosthenes)
  assumes: 0 <= x <= 5,000,000; 'less than N' counts primes up to N - 1
```

What the worked example means: This calculation determines the number of prime numbers less than or equal to the input bound 99. Applying the sieve of Eratosthenes identifies all primes within this range, and tallying them produces the result 25. This value represents the cumulative density of primes up to that specific limit.

Square roots modulo a prime — `sqrt_mod`

Every x in 0..p-1 with x^2 ≡ a (mod p), for a prime p.

$$x^2 \equiv a \pmod{p}, \quad a^{(p-1)/2} \equiv 1$$

symbol		meaning	SI unit
`a`	in	the integer a	—
`p`	in	a prime modulus	—
`roots`	out	all square roots of a modulo p, ascending	—
`count`	out	number of roots (0, 1 or 2)	—
`smallest`	out	the smallest root (none when a is a non-residue)	—

Applies when: p prime.

```
$ discrete-math solve sqrt_mod a=10 p=13
roots = [6,7]
count = 2
smallest = 6
  by: Square roots modulo a prime  (Euler's criterion decides solvability; Tonelli-Shanks finds a
root r, the other is p - r)
  assumes: p prime
```

What the worked example means: This calculation determines all integers x such that x squared is congruent to 10 modulo 13. It first verifies solvability using Euler's criterion, then applies the Tonelli-Shanks

algorithm to find the principal root. The result includes the set of valid roots [6, 7], their quantity 2, and the smallest value among them 6. If no solution exists, the count is zero and the root list is empty.

Sums of two squares — `two_squares`

Every way to write n = x^2 + y^2 with 0 <= x <= y, and r2(n), the number of ordered signed representations.

$$r_2(n) = 4\big(d_1(n) - d_3(n)\big)$$

symbol		meaning	SI unit
`n`	in	a whole number (0..10^12)	—
`representations`	out	pairs [x, y] with 0 <= x <= y and x^2 + y^2 = n	—
`count`	out	number of such pairs	—
`r2`	out	r2(n): ordered representations with signs	—

Applies when: 0 <= n <= 10^12.

```
$ discrete-math solve two_squares n=325
representations = [[1,18],[6,17],[10,15]]
count = 3
r2 = 24
  by: Sums of two squares  (search x up to sqrt(n/2) with an exact integer square root; r2(n) = 4
(d1(n) - d3(n)) from the divisors)
  assumes: 0 <= n <= 10^12
```

What the worked example means: This calculation identifies all pairs of integers where the sum of their squares equals the input number 325. The result lists these pairs as [[1, 18], [6, 17], [10, 15]], ensuring that each pair is ordered with the first number less than or equal to the second. The field 3 reports the number of such distinct unordered pairs found. Additionally, 24 provides the total number of representations when considering both order and signs, calculated using the divisor formula. This method efficiently determines the structure of the number in terms of quadratic forms.

The n-th prime — `nth_prime`

The n-th prime number p_n (p_1 = 2).

$$p_n < n(\ln n + \ln\ln n) \quad (n \geq 6)$$

symbol		meaning	SI unit
`n`	in	the index n (1..300,000)	—
`prime`	out	the n-th prime	—

Applies when: 1 <= n <= 300,000; p_1 = 2.

```
$ discrete-math solve nth_prime n=100
prime = 541
  by: The n-th prime  (sieve of Eratosthenes up to the Rosser bound n (ln n + ln ln n))
  assumes: 1 <= n <= 300,000; p_1 = 2
```

What the worked example means: This calculation identifies the specific prime number located at position 100 in the sequence of all primes. It uses the Sieve of Eratosthenes, limiting the search space to the Rosser bound, which guarantees that the 100-th prime is less than 100 times the sum of the natural logarithm of 100 and the natural logarithm of the natural logarithm of 100. The result, 541, is the exact integer value of that prime.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.